Lecture 5: Sequence Matching

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E A G A W G H E - E

- - P - A W - H E A E

Global matching, dynamic programming, global sequence matching

Outline

1. **Why Align**
2. Dynamic Programming
3. Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

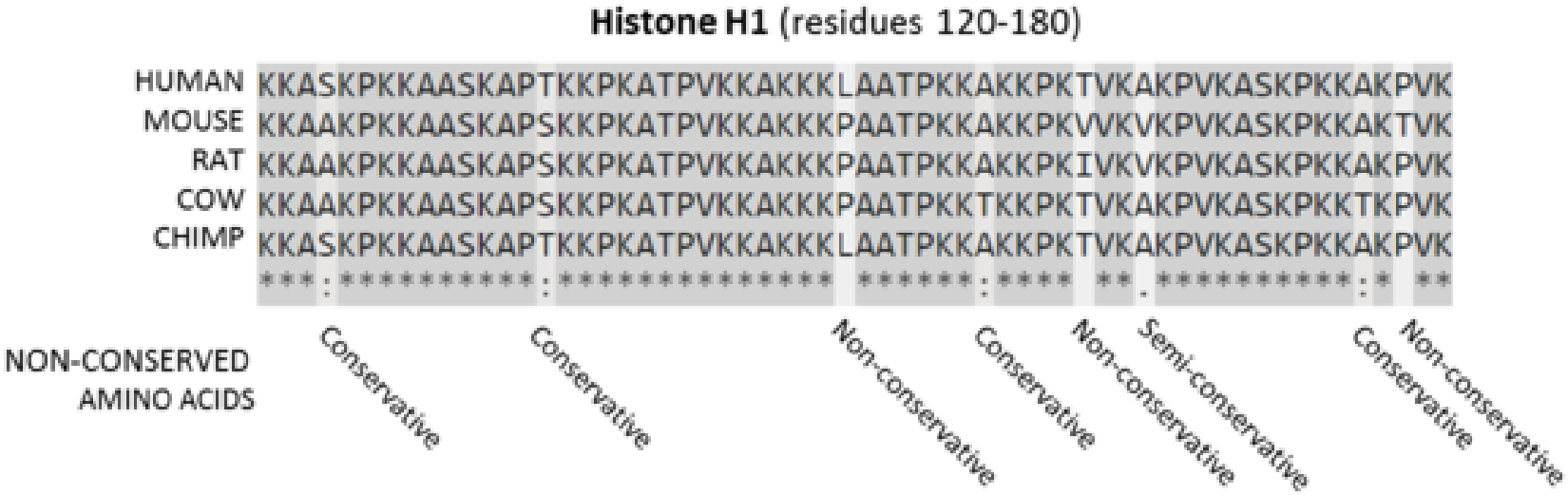
$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E A G A W G H E - E

- - P - A W - H E A E

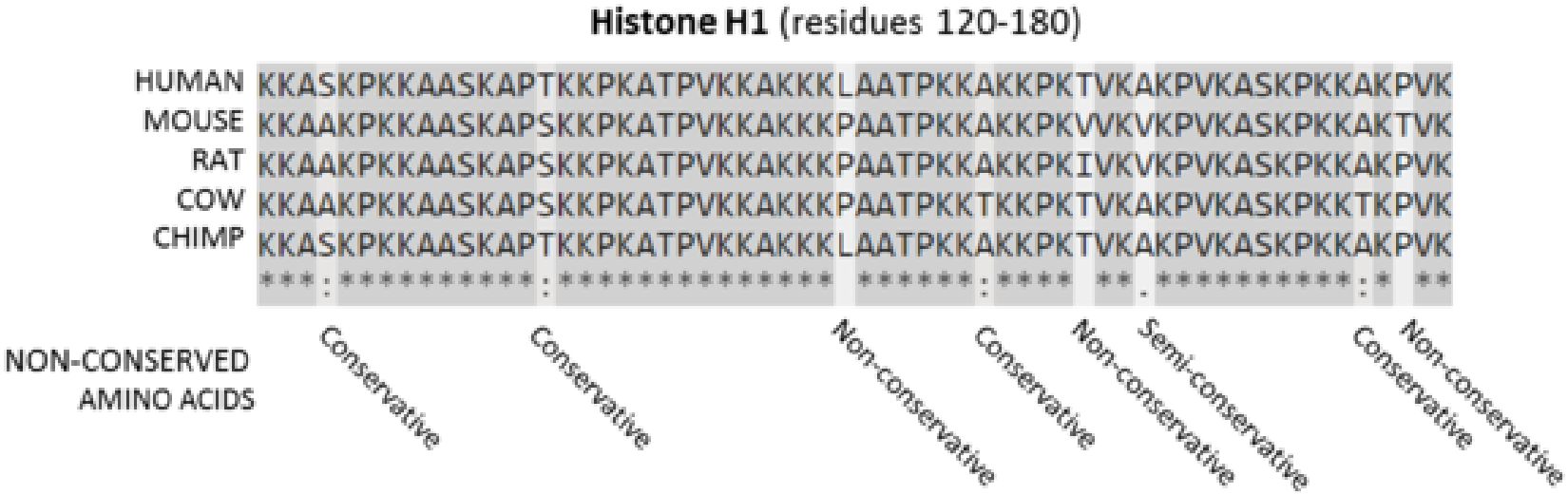
Alignment and Evolution

- We want to transfer our knowledge about the function of a protein though knowledge of similar proteins in different organisms
- Most proteins found in an organism are close to those of near relatives and sometimes not so near relatives



Alignment and Evolution

- We want to transfer our knowledge about the function of a protein though knowledge of similar proteins in different organisms
- Most proteins found in an organism are close to those of near relatives and sometimes not so near relatives



Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Practical Alignment

- Alignment with known genomes is now the first task after sequencing
- It is done with both DNA and protein
- The mutation rate is sufficiently high that we would expect no alignment between most species except if there is a selective pressure to conserve the structure
- For DNA coding proteins there are a number of "synonymous mutations" that leave the protein unchanged—DNA is less conserved than proteins
- In proteins some amino acids behave similarly so there is less selection pressure against mutations between them

Synonymous Mutations

Codon Usage Table

Codon Usage: The Bacterial and Plant Plastid Code (transl_table=11)

CCA(P) 0.84%	CGA(R) 0.35%	CAA(Q) 1.54%	CTA(L) 0.39%
CCG(P) 2.34%	CGG(R) 0.54%	CAG(Q) 2.90%	CTG(L) 5.33%
CCT(P) 0.69%	CGT(R) 2.10%	CAT(H) 1.28%	CTT(L) 1.10%
CCC(P) 0.55%	CGC(R) 2.22%	CAC(H) 0.96%	CTC(L) 1.12%
GCA(A) 2.03%	GGA(G) 0.79%	GAA(E) 3.97%	GTA(V) 1.09%
GCG(A) 3.40%	GGG(G) 1.10%	GAG(E) 1.79%	GTG(V) 2.63%
GCT(A) 1.53%	GGT(G) 2.48%	GAT(D) 3.21%	GTT(V) 1.83%
GCC(A) 2.57%	GGC(G) 2.97%	GAC(D) 1.91%	GTC(V) 1.53%
ACA(T) 0.69%	AGA(R) 0.20%	AAA(K) 3.36%	ATA(I) 0.42%
ACG(T) 1.44%	AGG(R) 0.11%	AAG(K) 1.03%	ATG(M) 2.79%
ACT(T) 0.89%	AGT(S) 0.86%	AAT(N) 1.74%	ATT(I) 3.05%
ACC(T) 2.34%	AGC(S) 1.60%	AAC(N) 2.14%	ATC(I) 2.52%
TCA(S) 0.70%	TGA(*) 0.09%	TAA(*) 0.20%	TTA(L) 1.38%
TCG(S) 0.89%	TGG(W) 1.54%	TAG(*) 0.02%	TTG(L) 1.37%
TCT(S) 0.84%	TGT(C) 0.51%	TAT(Y) 1.61%	TTT(F) 2.23%
TCC(S) 0.86%	TGC(C) 0.64%	TAC(Y) 1.23%	TTC(F) 1.66%

- Some amino acids are rarer than others
- But some codon sequences are slightly more popular than their synonymous partners
- They may be more robust to mutation
- They may be slight biases due to the translation, etc.

Synonymous Mutations

Codon Usage Table

Codon Usage: The Bacterial and Plant Plastid Code (transl_table=11)

CCA(P) 0.84%	CGA(R) 0.35%	CAA(Q) 1.54%	CTA(L) 0.39%
CCG(P) 2.34%	CGG(R) 0.54%	CAG(Q) 2.90%	CTG(L) 5.33%
CCT(P) 0.69%	CGT(R) 2.10%	CAT(H) 1.28%	CTT(L) 1.10%
CCC(P) 0.55%	CGC(R) 2.22%	CAC(H) 0.96%	CTC(L) 1.12%
GCA(A) 2.03%	GGA(G) 0.79%	GAA(E) 3.97%	GTA(V) 1.09%
GCG(A) 3.40%	GGG(G) 1.10%	GAG(E) 1.79%	GTG(V) 2.63%
GCT(A) 1.53%	GGT(G) 2.48%	GAT(D) 3.21%	GTT(V) 1.83%
GCC(A) 2.57%	GGC(G) 2.97%	GAC(D) 1.91%	GTC(V) 1.53%
ACA(T) 0.69%	AGA(R) 0.20%	AAA(K) 3.36%	ATA(I) 0.42%
ACG(T) 1.44%	AGG(R) 0.11%	AAG(K) 1.03%	ATG(M) 2.79%
ACT(T) 0.89%	AGT(S) 0.86%	AAT(N) 1.74%	ATT(I) 3.05%
ACC(T) 2.34%	AGC(S) 1.60%	AAC(N) 2.14%	ATC(I) 2.52%
TCA(S) 0.70%	TGA(*) 0.09%	TAA(*) 0.20%	TTA(L) 1.38%
TCG(S) 0.89%	TGG(W) 1.54%	TAG(*) 0.02%	TTG(L) 1.37%
TCT(S) 0.84%	TGT(C) 0.51%	TAT(Y) 1.61%	TTT(F) 2.23%
TCC(S) 0.86%	TGC(C) 0.64%	TAC(Y) 1.23%	TTC(F) 1.66%

- Some amino acids are rarer than others
- But some codon sequences are slightly more popular than their synonymous partners
- They may be more robust to mutation
- They may be slight biases due to the translation, etc.

Synonymous Mutations

Codon Usage Table

Codon Usage: The Bacterial and Plant Plastid Code (transl_table=11)

CCA(P) 0.84%	CGA(R) 0.35%	CAA(Q) 1.54%	CTA(L) 0.39%
CCG(P) 2.34%	CGG(R) 0.54%	CAG(Q) 2.90%	CTG(L) 5.33%
CCT(P) 0.69%	CGT(R) 2.10%	CAT(H) 1.28%	CTT(L) 1.10%
CCC(P) 0.55%	CGC(R) 2.22%	CAC(H) 0.96%	CTC(L) 1.12%
GCA(A) 2.03%	GGA(G) 0.79%	GAA(E) 3.97%	GTA(V) 1.09%
GCG(A) 3.40%	GGG(G) 1.10%	GAG(E) 1.79%	GTG(V) 2.63%
GCT(A) 1.53%	GGT(G) 2.48%	GAT(D) 3.21%	GTT(V) 1.83%
GCC(A) 2.57%	GGC(G) 2.97%	GAC(D) 1.91%	GTC(V) 1.53%
ACA(T) 0.69%	AGA(R) 0.20%	AAA(K) 3.36%	ATA(I) 0.42%
ACG(T) 1.44%	AGG(R) 0.11%	AAG(K) 1.03%	ATG(M) 2.79%
ACT(T) 0.89%	AGT(S) 0.86%	AAT(N) 1.74%	ATT(I) 3.05%
ACC(T) 2.34%	AGC(S) 1.60%	AAC(N) 2.14%	ATC(I) 2.52%
TCA(S) 0.70%	TGA(*) 0.09%	TAA(*) 0.20%	TTA(L) 1.38%
TCG(S) 0.89%	TGG(W) 1.54%	TAG(*) 0.02%	TTG(L) 1.37%
TCT(S) 0.84%	TGT(C) 0.51%	TAT(Y) 1.61%	TTT(F) 2.23%
TCC(S) 0.86%	TGC(C) 0.64%	TAC(Y) 1.23%	TTC(F) 1.66%

- Some amino acids are rarer than others
- But some codon sequences are slightly more popular than their synonymous partners
- They may be more robust to mutation
- They may be slight biases due to the translation, etc.

Synonymous Mutations

Codon Usage Table

Codon Usage: The Bacterial and Plant Plastid Code (transl_table=11)

CCA(P) 0.84%	CGA(R) 0.35%	CAA(Q) 1.54%	CTA(L) 0.39%
CCG(P) 2.34%	CGG(R) 0.54%	CAG(Q) 2.90%	CTG(L) 5.33%
CCT(P) 0.69%	CGT(R) 2.10%	CAT(H) 1.28%	CTT(L) 1.10%
CCC(P) 0.55%	CGC(R) 2.22%	CAC(H) 0.96%	CTC(L) 1.12%
GCA(A) 2.03%	GGA(G) 0.79%	GAA(E) 3.97%	GTA(V) 1.09%
GCG(A) 3.40%	GGG(G) 1.10%	GAG(E) 1.79%	GTG(V) 2.63%
GCT(A) 1.53%	GGT(G) 2.48%	GAT(D) 3.21%	GTT(V) 1.83%
GCC(A) 2.57%	GGC(G) 2.97%	GAC(D) 1.91%	GTC(V) 1.53%
ACA(T) 0.69%	AGA(R) 0.20%	AAA(K) 3.36%	ATA(I) 0.42%
ACG(T) 1.44%	AGG(R) 0.11%	AAG(K) 1.03%	ATG(M) 2.79%
ACT(T) 0.89%	AGT(S) 0.86%	AAT(N) 1.74%	ATT(I) 3.05%
ACC(T) 2.34%	AGC(S) 1.60%	AAC(N) 2.14%	ATC(I) 2.52%
TCA(S) 0.70%	TGA(*) 0.09%	TAA(*) 0.20%	TTA(L) 1.38%
TCG(S) 0.89%	TGG(W) 1.54%	TAG(*) 0.02%	TTG(L) 1.37%
TCT(S) 0.84%	TGT(C) 0.51%	TAT(Y) 1.61%	TTT(F) 2.23%
TCC(S) 0.86%	TGC(C) 0.64%	TAC(Y) 1.23%	TTC(F) 1.66%

- Some amino acids are rarer than others
- But some codon sequences are slightly more popular than their synonymous partners
- They may be more robust to mutation
- They may be slight biases due to the translation, etc.

Amino Acid Similarity

- For amino acids the most important distinguishing property is whether they are **hydrophobic** (oily) or **hydrophilic** (charged)
- There are other properties such as available hydrogen bonds, etc.
- From many observation we can compute a table giving the log-odds of one amino acid being substituted by another
- Two commonly used substitution tables (matrices) are PAM and BLOSUM
- Likely substitutions are positive, unlikely ones negative

Amino Acid Similarity

- For amino acids the most important distinguishing property is whether they are **hydrophobic** (oily) or **hydrophilic** (charged)
- There are other properties such as available hydrogen bonds, etc.
- From many observation we can compute a table giving the log-odds of one amino acid being substituted by another
- Two commonly used substitution tables (matrices) are PAM and BLOSUM
- Likely substitutions are positive, unlikely ones negative

Amino Acid Similarity

- For amino acids the most important distinguishing property is whether they are **hydrophobic** (oily) or **hydrophilic** (charged)
- There are other properties such as available hydrogen bonds, etc.
- From many observation we can compute a table giving the log-odds of one amino acid being substituted by another
- Two commonly used substitution tables (matrices) are PAM and BLOSUM
- Likely substitutions are positive, unlikely ones negative

Amino Acid Similarity

- For amino acids the most important distinguishing property is whether they are **hydrophobic** (oily) or **hydrophilic** (charged)
- There are other properties such as available hydrogen bonds, etc.
- From many observation we can compute a table giving the log-odds of one amino acid being substituted by another
- Two commonly used substitution tables (matrices) are PAM and BLOSUM
- Likely substitutions are positive, unlikely ones negative

Amino Acid Similarity

- For amino acids the most important distinguishing property is whether they are **hydrophobic** (oily) or **hydrophilic** (charged)
- There are other properties such as available hydrogen bonds, etc.
- From many observation we can compute a table giving the log-odds of one amino acid being substituted by another
- Two commonly used substitution tables (matrices) are PAM and BLOSUM
- Likely substitutions are positive, unlikely ones negative

Blosum50

	A	R	N	D	C	Q	E	G	H	I	L	K	M	F	P	S	T	W	Y	V
A	**5**	−2	−1	−2	−1	−1	−1	0	−2	−1	−2	−1	−1	−3	−1	1	0	−3	−2	0
R	−2	**7**	−1	−2	−4	1	0	−3	0	−4	−3	3	−2	−3	−3	−1	−1	−3	−1	−3
N	−1	−1	**7**	2	−2	0	0	0	1	−3	−4	0	−2	−4	−2	1	0	−4	−2	−3
D	−2	−2	2	**8**	−4	0	2	−1	−1	−4	−4	−1	−4	−5	−1	0	−1	−5	−3	−4
C	−1	−4	−2	−4	**13**	−3	−3	−3	−3	−2	−2	−3	−2	−2	−4	−1	−1	−5	−3	−1
Q	−1	1	0	0	−3	**7**	2	−2	1	−3	−2	2	0	−4	−1	0	−1	−1	−1	−3
E	−1	0	0	2	−3	2	**6**	−3	0	−4	−3	1	−2	−3	−1	−1	−1	−3	−2	−3
G	0	−3	0	−1	−3	−2	−3	**8**	−2	−4	−4	−2	−3	−4	−2	0	−2	−3	−3	−4
H	−2	0	1	−1	−3	1	0	−2	**10**	−4	−3	0	−1	−1	−2	−1	−2	−3	2	−4
I	−1	−4	−3	−4	−2	−3	−4	−4	−4	**5**	2	−3	2	0	−3	−3	−1	−3	−1	4
L	−2	−3	−4	−4	−2	−2	−3	−4	−3	2	**5**	−3	3	1	−4	−3	−1	−2	−1	1
K	−1	3	0	−1	−3	2	1	−2	0	−3	−3	**6**	−2	−4	−1	0	−1	−3	−2	−3
M	−1	−2	−2	−4	−2	0	−2	−3	−1	2	3	−2	**7**	0	−3	−2	−1	−1	0	1
F	−3	−3	−4	−5	−2	−4	−3	−4	−1	0	1	−4	0	**8**	−4	−3	−2	1	4	−1
P	−1	−3	−2	−1	−4	−1	−1	−2	−2	−3	−4	−1	−3	−4	**10**	−1	−1	−4	−3	−3
S	1	−1	1	0	−1	0	−1	0	−1	−3	−3	0	−2	−3	−1	**5**	2	−4	−2	−2
T	0	−1	0	−1	−1	−1	−1	−2	−2	−1	−1	−1	−1	−2	−1	2	**5**	−3	−2	0
W	−3	−3	−4	−5	−5	−1	−3	−3	−3	−3	−2	−3	−1	1	−4	−4	−3	**15**	2	−3
Y	−2	−1	−2	−3	−3	−1	−2	−3	2	−1	−1	−2	0	4	−3	−2	−2	2	**8**	−1
V	0	−3	−3	−4	−1	−3	−3	−4	−4	4	1	−3	1	−1	−3	−2	0	−3	−1	**5**

Gaps

- It is not infrequent that we find good matches where there has been an insertion or deletion of some part of the string
- We start by considering a linear cost where the cost of a gap is proportional to its length
- In biology, a linear cost model is a poor description of reality
- Where there is an insertion, then it is often quite long (this is a consequence of the copy mechanism in cells)
- We return to dealing with long gaps in the next lecture

Gaps

- It is not infrequent that we find good matches where there has been an insertion or deletion of some part of the string
- We start by considering a linear cost where the cost of a gap is proportional to its length
- In biology, a linear cost model is a poor description of reality
- Where there is an insertion, then it is often quite long (this is a consequence of the copy mechanism in cells)
- We return to dealing with long gaps in the next lecture

Gaps

- It is not infrequent that we find good matches where there has been an insertion or deletion of some part of the string
- We start by considering a linear cost where the cost of a gap is proportional to its length
- In biology, a linear cost model is a poor description of reality
- Where there is an insertion, then it is often quite long (this is a consequence of the copy mechanism in cells)
- We return to dealing with long gaps in the next lecture

Gaps

- It is not infrequent that we find good matches where there has been an insertion or deletion of some part of the string
- We start by considering a linear cost where the cost of a gap is proportional to its length
- In biology, a linear cost model is a poor description of reality
- Where there is an insertion, then it is often quite long (this is a consequence of the copy mechanism in cells)
- We return to dealing with long gaps in the next lecture

Gaps

- It is not infrequent that we find good matches where there has been an insertion or deletion of some part of the string
- We start by considering a linear cost where the cost of a gap is proportional to its length
- In biology, a linear cost model is a poor description of reality
- Where there is an insertion, then it is often quite long (this is a consequence of the copy mechanism in cells)
- We return to dealing with long gaps in the next lecture

Inexact Sequence Matching

- Given two sequences we want to find the best match between them
- We can measure this distance in terms of an **edit distance** or **Levenshtein** distance which measures the minimum number of moves to go from one sequence to another
- There are two cases that are commonly considered
  - ⋆ Global matching: where we match the two complete strings
  - ⋆ Local matching: we find the sub-string with the best match (we treat this in the next lecture)

Inexact Sequence Matching

- Given two sequences we want to find the best match between them
- We can measure this distance in terms of an **edit distance** or **Levenshtein** distance which measures the minimum number of moves to go from one sequence to another
- There are two cases that are commonly considered
  - ⋆ Global matching: where we match the two complete strings
  - ⋆ Local matching: we find the sub-string with the best match (we treat this in the next lecture)

Inexact Sequence Matching

- Given two sequences we want to find the best match between them
- We can measure this distance in terms of an **edit distance** or **Levenshtein** distance which measures the minimum number of moves to go from one sequence to another
- There are two cases that are commonly considered
  - ⋆ Global matching: where we match the two complete strings
  - ⋆ Local matching: we find the sub-string with the best match (we treat this in the next lecture)

Inexact Sequence Matching

- Given two sequences we want to find the best match between them
- We can measure this distance in terms of an **edit distance** or **Levenshtein** distance which measures the minimum number of moves to go from one sequence to another
- There are two cases that are commonly considered
  - ⋆ Global matching: where we match the two complete strings
  - ⋆ Local matching: we find the sub-string with the best match (we treat this in the next lecture)

Outline

1. Why Align
2. **Dynamic Programming**
3. Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E A G A W G H E - E

- - P - A W - H E A E

Algorithms for Matching

- We need efficient algorithms for matching
- The number of possible inexact matches is exponential in the size of the string so we can't exhaustively try all matches
- Fortunately dynamical programming provides a relatively efficient algorithm
- Dynamic programming is very important to a huge number of sequence-based algorithms
- We first quickly remind you of dynamic programming

Algorithms for Matching

- We need efficient algorithms for matching
- The number of possible inexact matches is exponential in the size of the string so we can't exhaustively try all matches
- Fortunately dynamical programming provides a relatively efficient algorithm
- Dynamic programming is very important to a huge number of sequence-based algorithms
- We first quickly remind you of dynamic programming

Algorithms for Matching

- We need efficient algorithms for matching
- The number of possible inexact matches is exponential in the size of the string so we can't exhaustively try all matches
- Fortunately dynamical programming provides a relatively efficient algorithm
- Dynamic programming is very important to a huge number of sequence-based algorithms
- We first quickly remind you of dynamic programming

Algorithms for Matching

- We need efficient algorithms for matching
- The number of possible inexact matches is exponential in the size of the string so we can't exhaustively try all matches
- Fortunately dynamical programming provides a relatively efficient algorithm
- Dynamic programming is very important to a huge number of sequence-based algorithms
- We first quickly remind you of dynamic programming

Algorithms for Matching

- We need efficient algorithms for matching
- The number of possible inexact matches is exponential in the size of the string so we can't exhaustively try all matches
- Fortunately dynamical programming provides a relatively efficient algorithm
- Dynamic programming is very important to a huge number of sequence-based algorithms
- We first quickly remind you of dynamic programming

Dynamic Programming

- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

Dynamic Programming

- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

Dynamic Programming

- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

Dynamic Programming

- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

Dynamic Programming

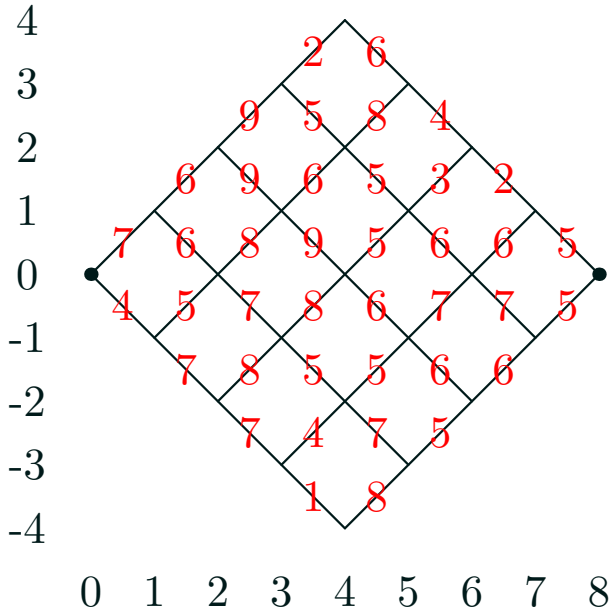
- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

Dynamic Programming

- One of the most powerful strategies for solving optimisation problems is **dynamic programming**
  - ⋆ Build a set of optimal partial solutions
  - ⋆ Increase the size of the partial solutions until you have a full solution
  - ⋆ Each step uses the set of optimal partial solutions found in the previous step
- Developed by Richard Bellman in the early 1950's
- The name is unfortunate as it doesn't have much to do with programming

A Toy Problem

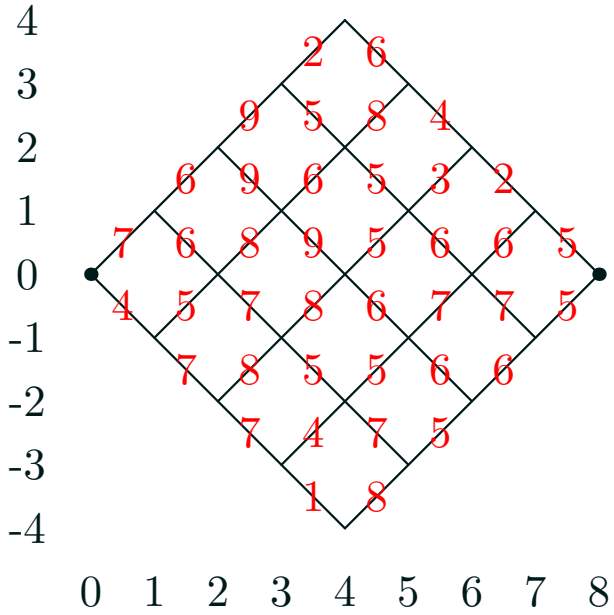
- Consider the problem of find a minimum cost path from point (0,0) to (8,0) on the lattice



- The costs of traversing each link is shown in red
- The cost of a path is the sum of weights on each link

A Toy Problem

- Consider the problem of find a minimum cost path from point (0,0) to (8,0) on the lattice

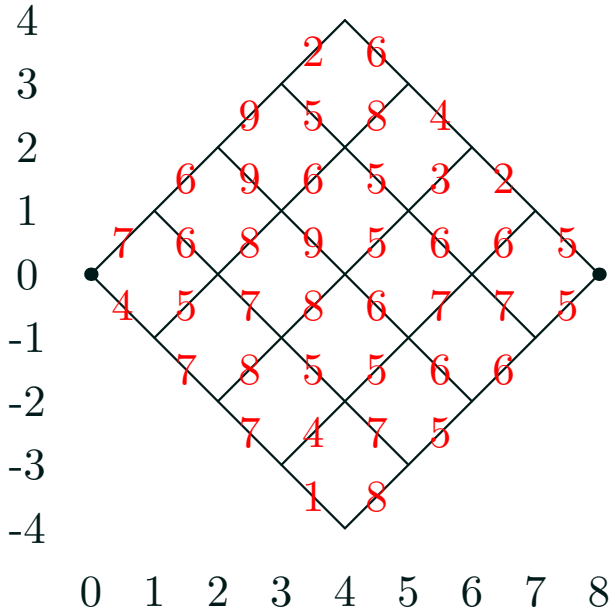


- The costs of traversing each link is shown in red

- The cost of a path is the sum of weights on each link

A Toy Problem

- Consider the problem of find a minimum cost path from point (0,0) to (8,0) on the lattice



- The costs of traversing each link is shown in red

- The cost of a path is the sum of weights on each link

Brute Force

- The obvious brute force strategy is to try every path
- For a problem with n steps we require $n/2$ to be diagonally up and $n/2$ to be diagonally down
- The total number of paths is

$$\binom{n}{n/2} \approx \sqrt{\frac{2}{\pi\, n}}\, 2^n$$

- For the problem shown above with $n = 8$ there are 70 paths
- For a problem with $n = 100$ there are 1.01×10^{29} paths

Brute Force

- The obvious brute force strategy is to try every path
- For a problem with n steps we require $n/2$ to be diagonally up and $n/2$ to be diagonally down
- The total number of paths is

$$\binom{n}{n/2} \approx \sqrt{\frac{2}{\pi\, n}}\, 2^n$$

- For the problem shown above with $n = 8$ there are 70 paths
- For a problem with $n = 100$ there are 1.01×10^{29} paths

Brute Force

- The obvious brute force strategy is to try every path
- For a problem with n steps we require $n/2$ to be diagonally up and $n/2$ to be diagonally down
- The total number of paths is

$$\binom{n}{n/2} \approx \sqrt{\frac{2}{\pi\, n}}\, 2^n$$

- For the problem shown above with $n = 8$ there are 70 paths
- For a problem with $n = 100$ there are 1.01×10^{29} paths

Brute Force

- The obvious brute force strategy is to try every path
- For a problem with n steps we require $n/2$ to be diagonally up and $n/2$ to be diagonally down
- The total number of paths is

$$\binom{n}{n/2} \approx \sqrt{\frac{2}{\pi\, n}}\, 2^n$$

- For the problem shown above with $n = 8$ there are 70 paths
- For a problem with $n = 100$ there are 1.01×10^{29} paths

Brute Force

- The obvious brute force strategy is to try every path
- For a problem with n steps we require $n/2$ to be diagonally up and $n/2$ to be diagonally down
- The total number of paths is

$$\binom{n}{n/2} \approx \sqrt{\frac{2}{\pi\, n}}\, 2^n$$

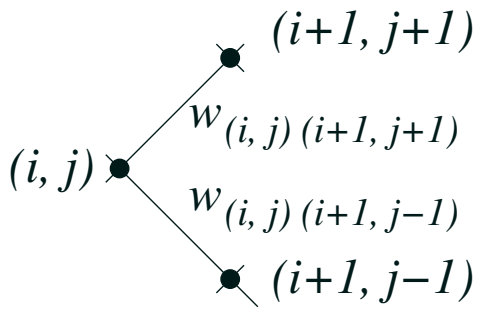
- For the problem shown above with $n = 8$ there are 70 paths
- For a problem with $n = 100$ there are 1.01×10^{29} paths

Building a Solution

- We can solve this problem efficiently using dynamic programming by considering optimal paths of shorter length

- Let $c_{(i,j)}$ denote the cost of the optimal path to node (i, j)

- We denote the weights between two points on the lattice by $w_{(i,j)(i+1,j\pm1)}$



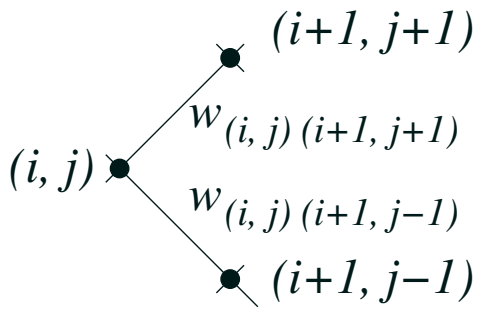
- We define the initial cost $c_{(0,0)} = 0$

Building a Solution

- We can solve this problem efficiently using dynamic programming by considering optimal paths of shorter length

- Let $c_{(i,j)}$ denote the cost of the optimal path to node (i, j)

- We denote the weights between two points on the lattice by $w_{(i,j)(i+1,j\pm1)}$



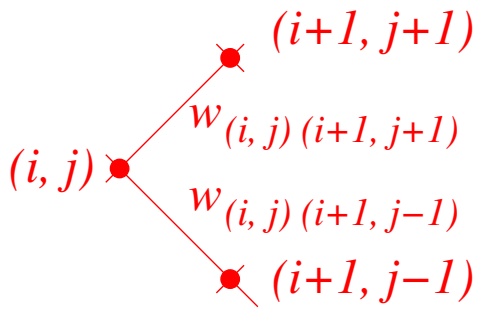
- We define the initial cost $c_{(0,0)} = 0$

Building a Solution

- We can solve this problem efficiently using dynamic programming by considering optimal paths of shorter length

- Let $c_{(i,j)}$ denote the cost of the optimal path to node (i, j)

- We denote the weights between two points on the lattice by $w_{(i,j)(i+1,j\pm1)}$



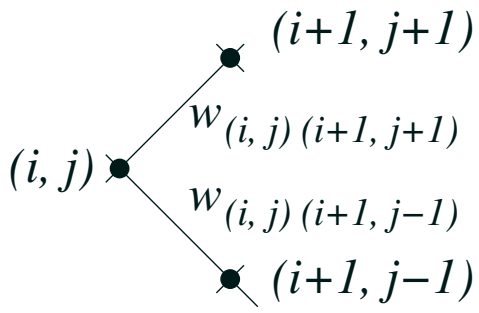
- We define the initial cost $c_{(0,0)} = 0$

Building a Solution

- We can solve this problem efficiently using dynamic programming by considering optimal paths of shorter length

- Let $c_{(i,j)}$ denote the cost of the optimal path to node (i, j)

- We denote the weights between two points on the lattice by $w_{(i,j)(i+1,j\pm1)}$



- We define the initial cost $c_{(0,0)} = 0$

Forward Algorithm

- Suppose we know the optimal costs for all the edge in column i
- Our task is to find the optimal cost at column $i+1$
- If we consider the sites in the lattice then the optimal cost will be

$$c_{(i+1,j)} = \min\Big(c_{(i,j+1)} + w_{(i,j+1)(i+1,j)},\ c_{(i,j-1)} + w_{(i,j-1)(i+1,j)}\Big)$$

- This is the defining equation in dynamic programming
- We have to treat the boundary sites specially, but this is just book-keeping

Forward Algorithm

- Suppose we know the optimal costs for all the edge in column i
- Our task is to find the optimal cost at column $i+1$
- If we consider the sites in the lattice then the optimal cost will be

$$c_{(i+1,j)} = \min\Big(c_{(i,j+1)} + w_{(i,j+1)(i+1,j)},\ c_{(i,j-1)} + w_{(i,j-1)(i+1,j)}\Big)$$

- This is the defining equation in dynamic programming
- We have to treat the boundary sites specially, but this is just book-keeping

Forward Algorithm

- Suppose we know the optimal costs for all the edge in column i
- Our task is to find the optimal cost at column $i+1$
- If we consider the sites in the lattice then the optimal cost will be

$$c_{(i+1,j)} = \min\Big(c_{(i,j+1)} + w_{(i,j+1)(i+1,j)},\ c_{(i,j-1)} + w_{(i,j-1)(i+1,j)}\Big)$$

- This is the defining equation in dynamic programming
- We have to treat the boundary sites specially, but this is just book-keeping

Forward Algorithm

- Suppose we know the optimal costs for all the edge in column i
- Our task is to find the optimal cost at column $i+1$
- If we consider the sites in the lattice then the optimal cost will be

$$c_{(i+1,j)} = \min\Big(c_{(i,j+1)} + w_{(i,j+1)(i+1,j)},\ c_{(i,j-1)} + w_{(i,j-1)(i+1,j)}\Big)$$

- This is the defining equation in dynamic programming
- We have to treat the boundary sites specially, but this is just book-keeping

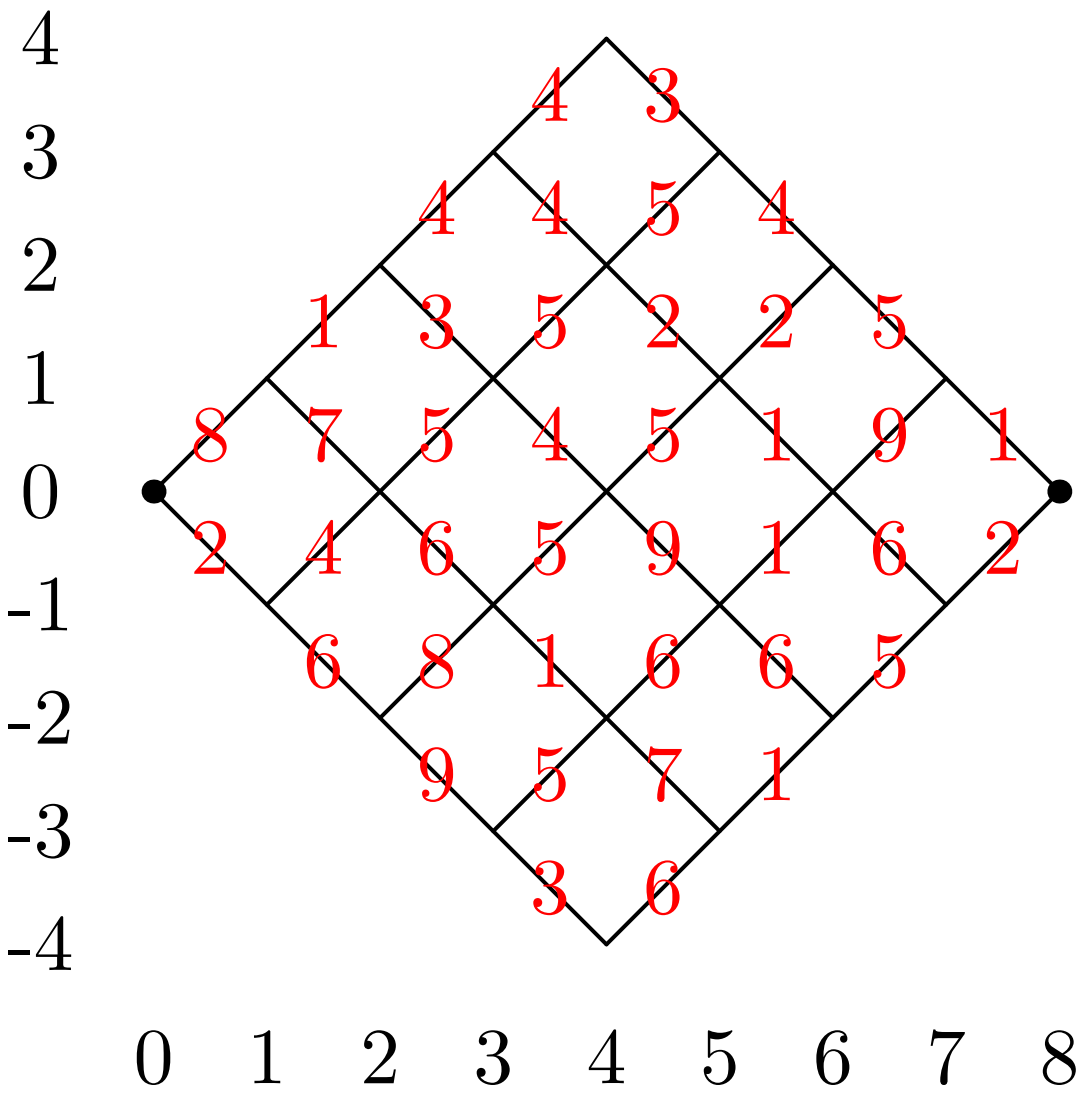
Forward Algorithm

- Suppose we know the optimal costs for all the edge in column i
- Our task is to find the optimal cost at column $i+1$
- If we consider the sites in the lattice then the optimal cost will be

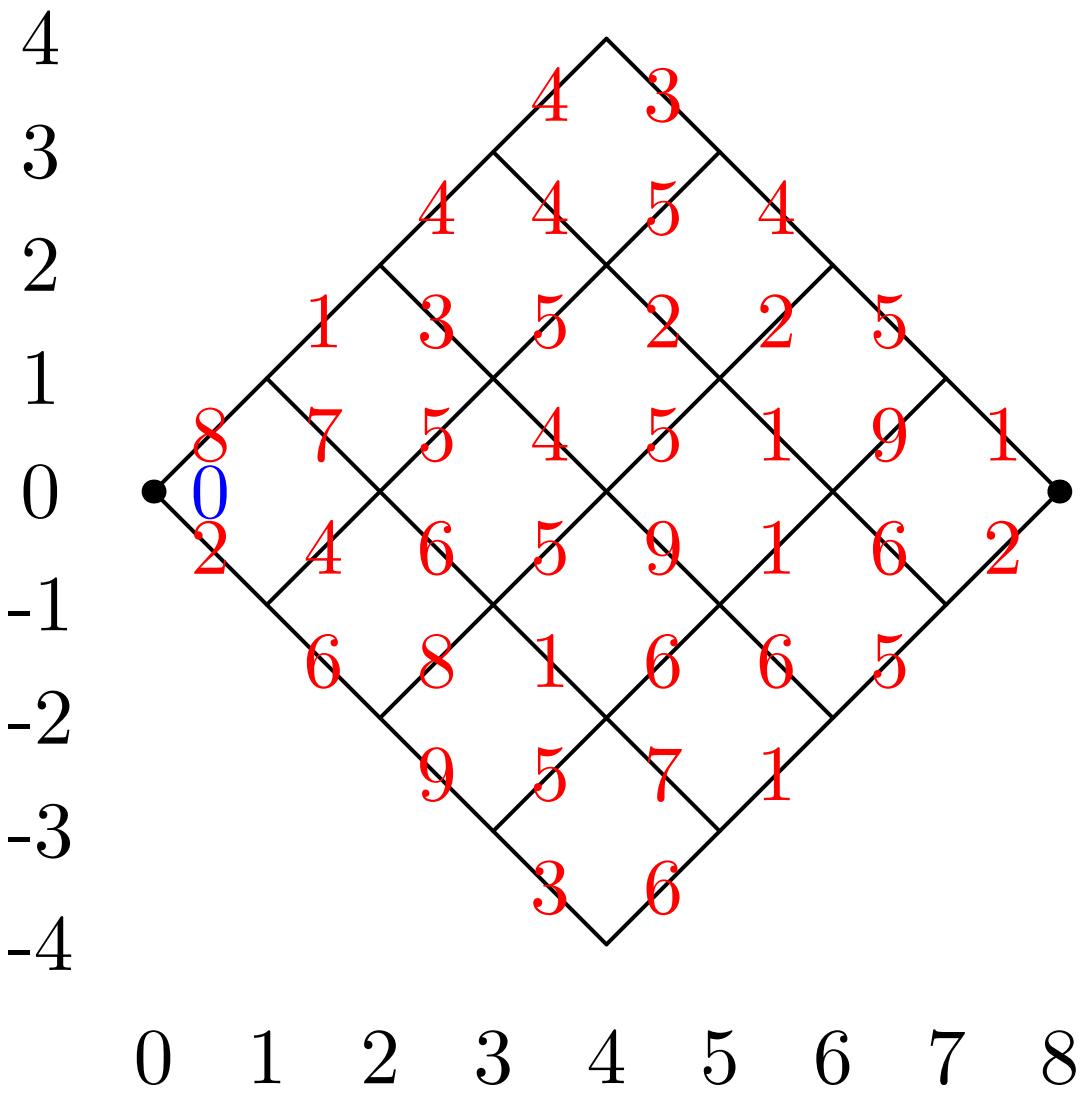
$$c_{(i+1,j)} = \min\Big(c_{(i,j+1)} + w_{(i,j+1)(i+1,j)},\ c_{(i,j-1)} + w_{(i,j-1)(i+1,j)}\Big)$$

- This is the defining equation in dynamic programming
- We have to treat the boundary sites specially, but this is just book-keeping

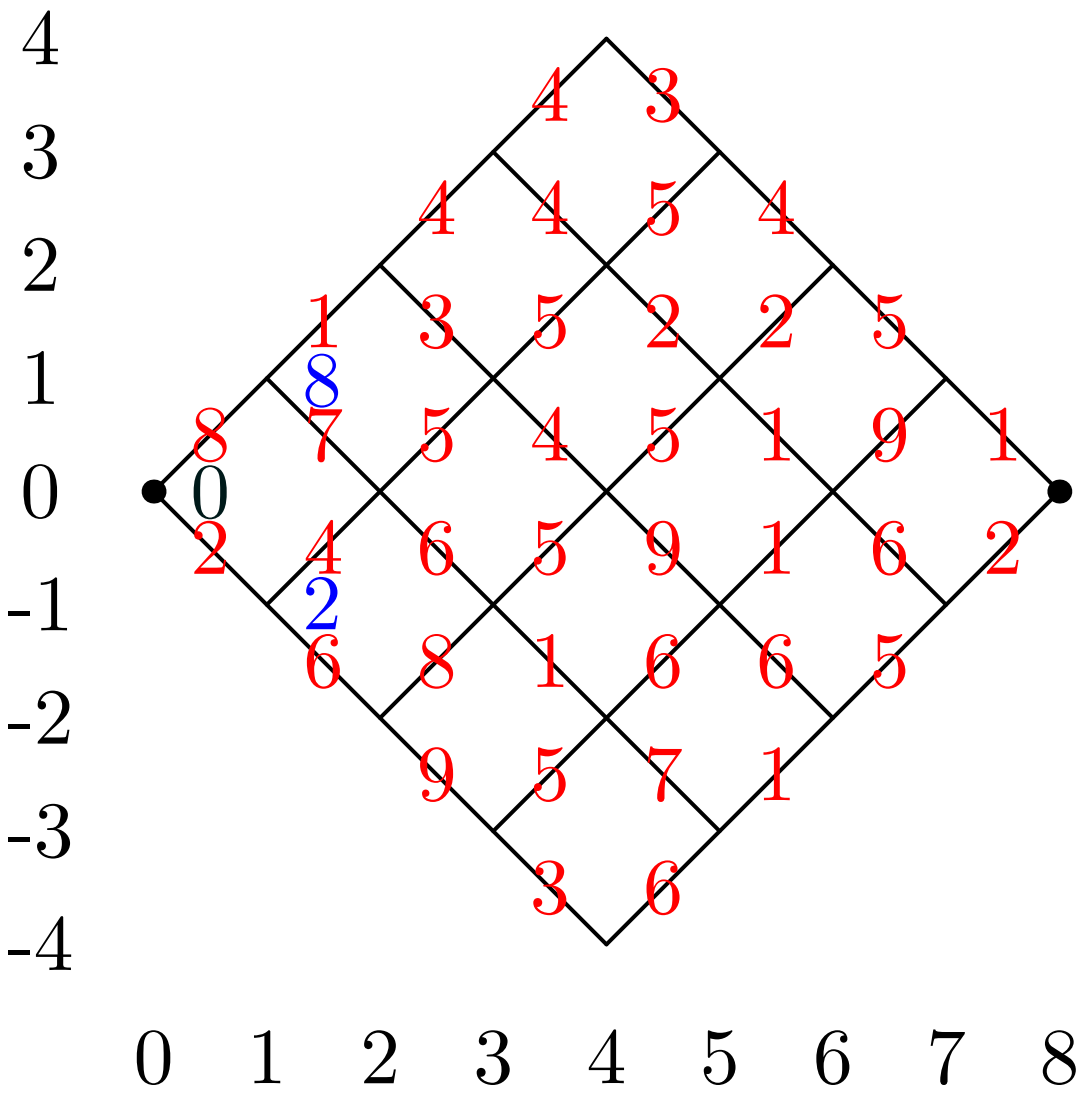
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
4 3
4 4 5 4
1 3 5 2 2 5
8 7 5 4 5 1 9 1
2 4 6 5 9 1 6 2
6 8 1 6 6 5
9 5 7 1
3 6

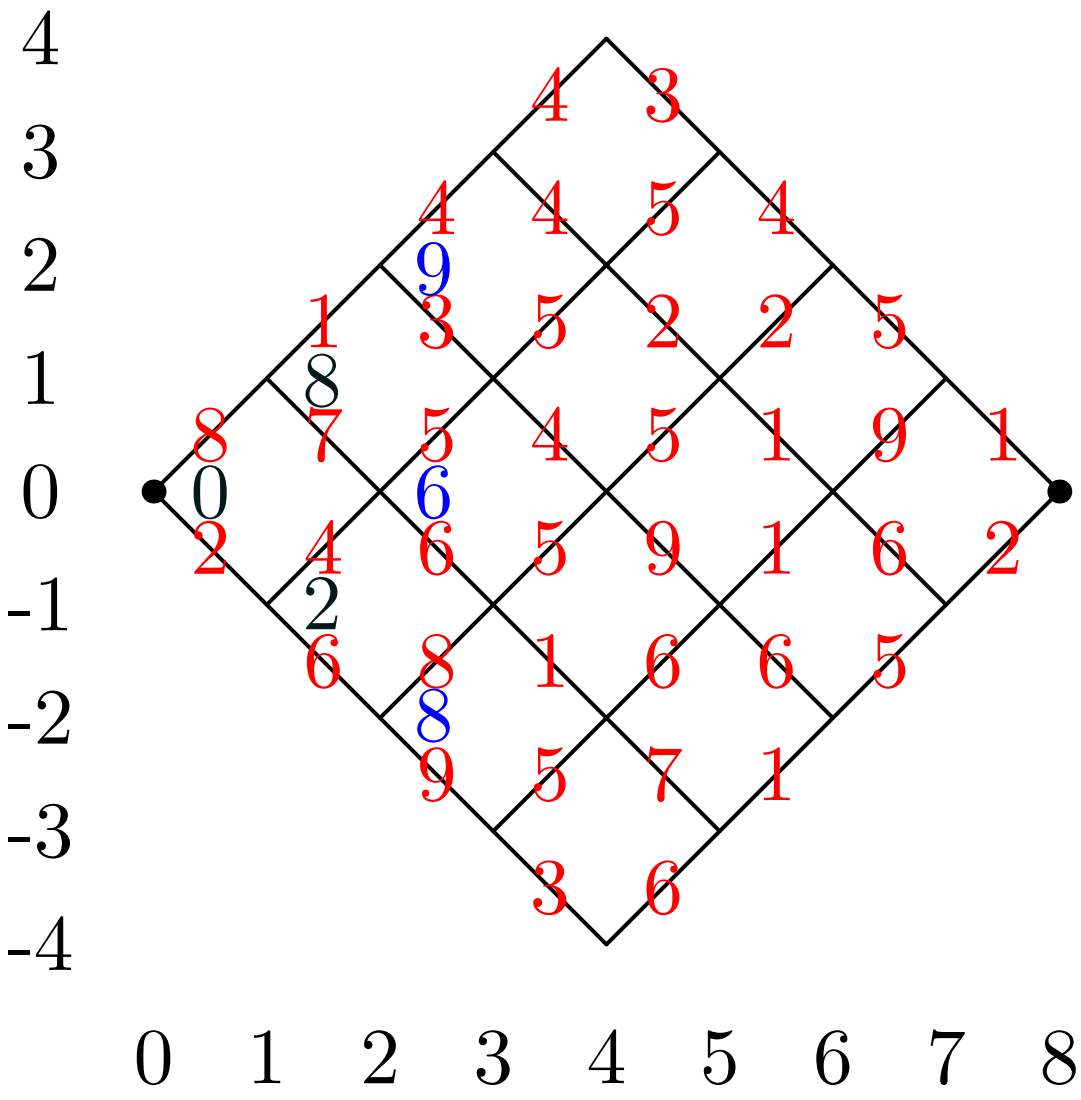
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
4 3
4 4 5 4
1 3 5 2 2 5
8 7 5 4 5 1 9 1
0
2 4 6 5 9 1 6 2
6 8 1 6 6 5
9 5 7 1
3 6

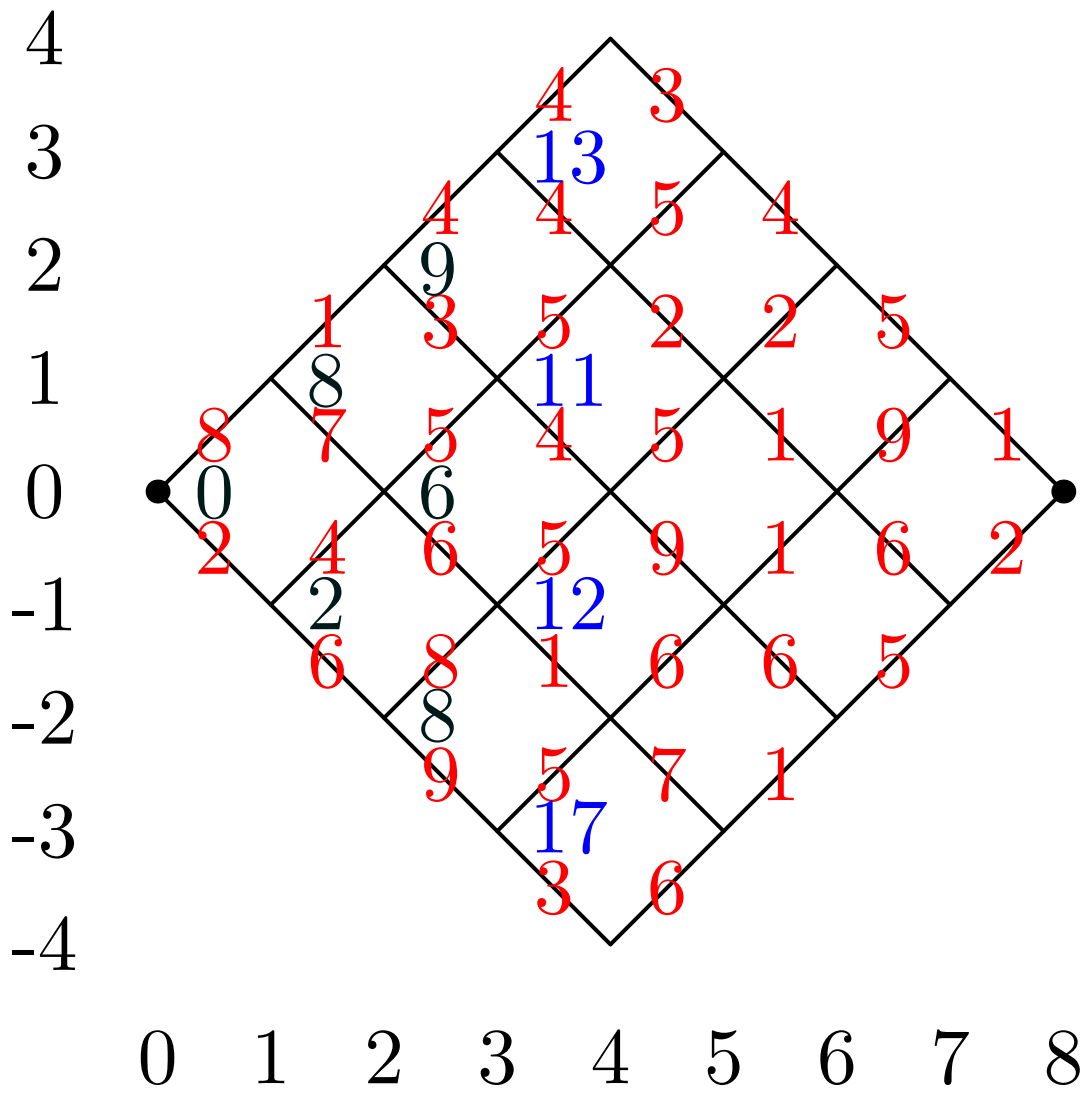
Example



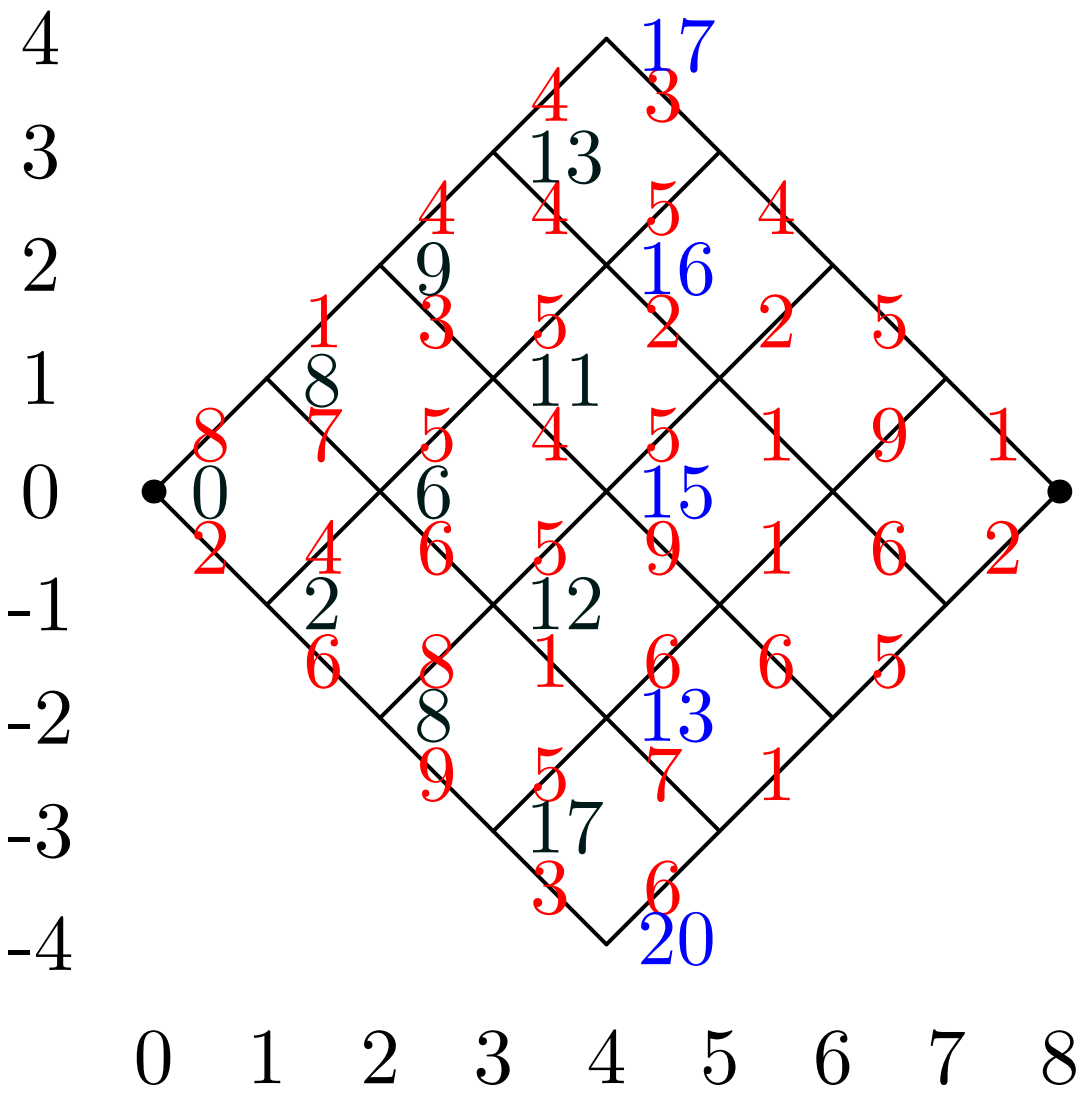
Example



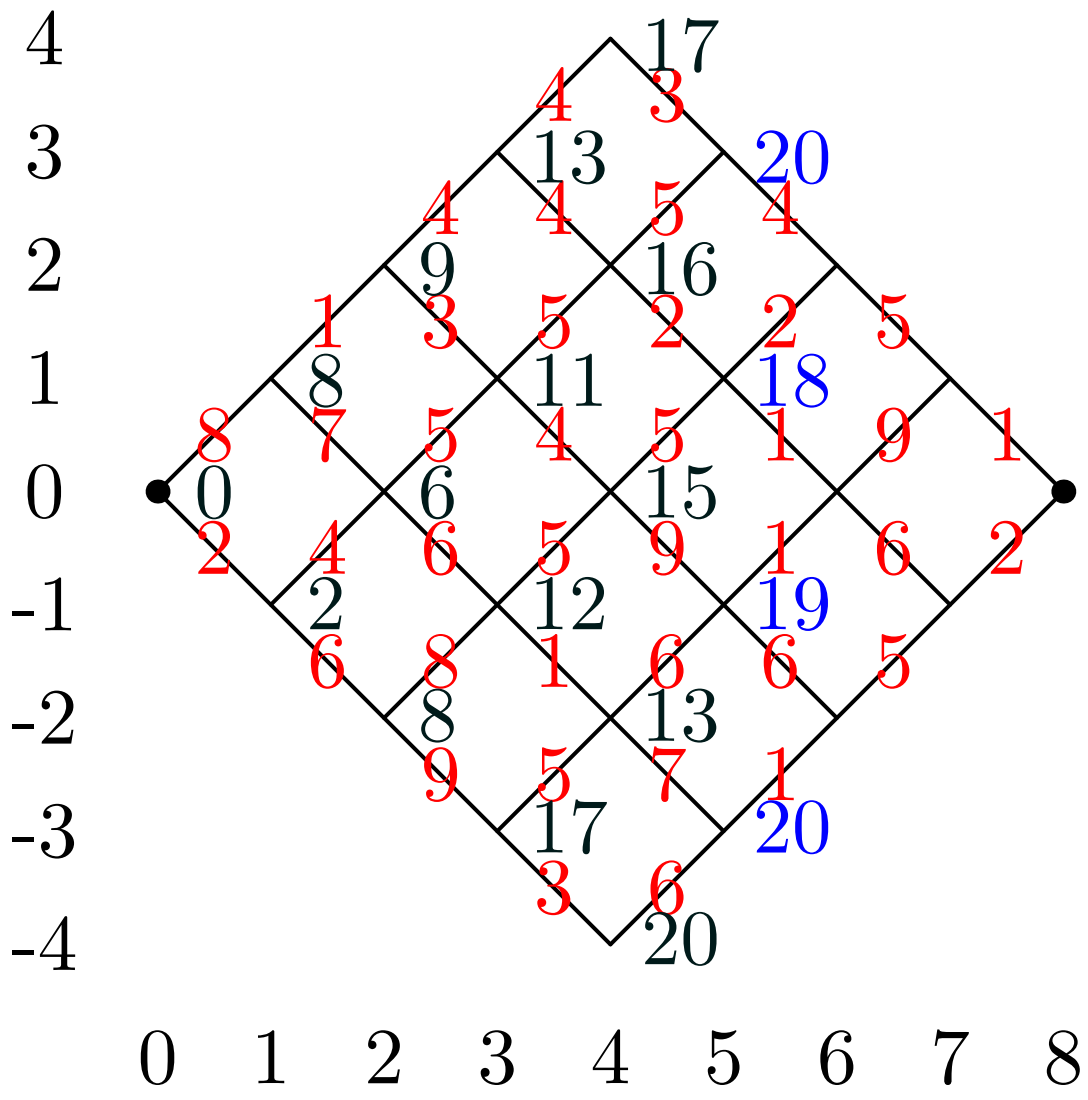
Example



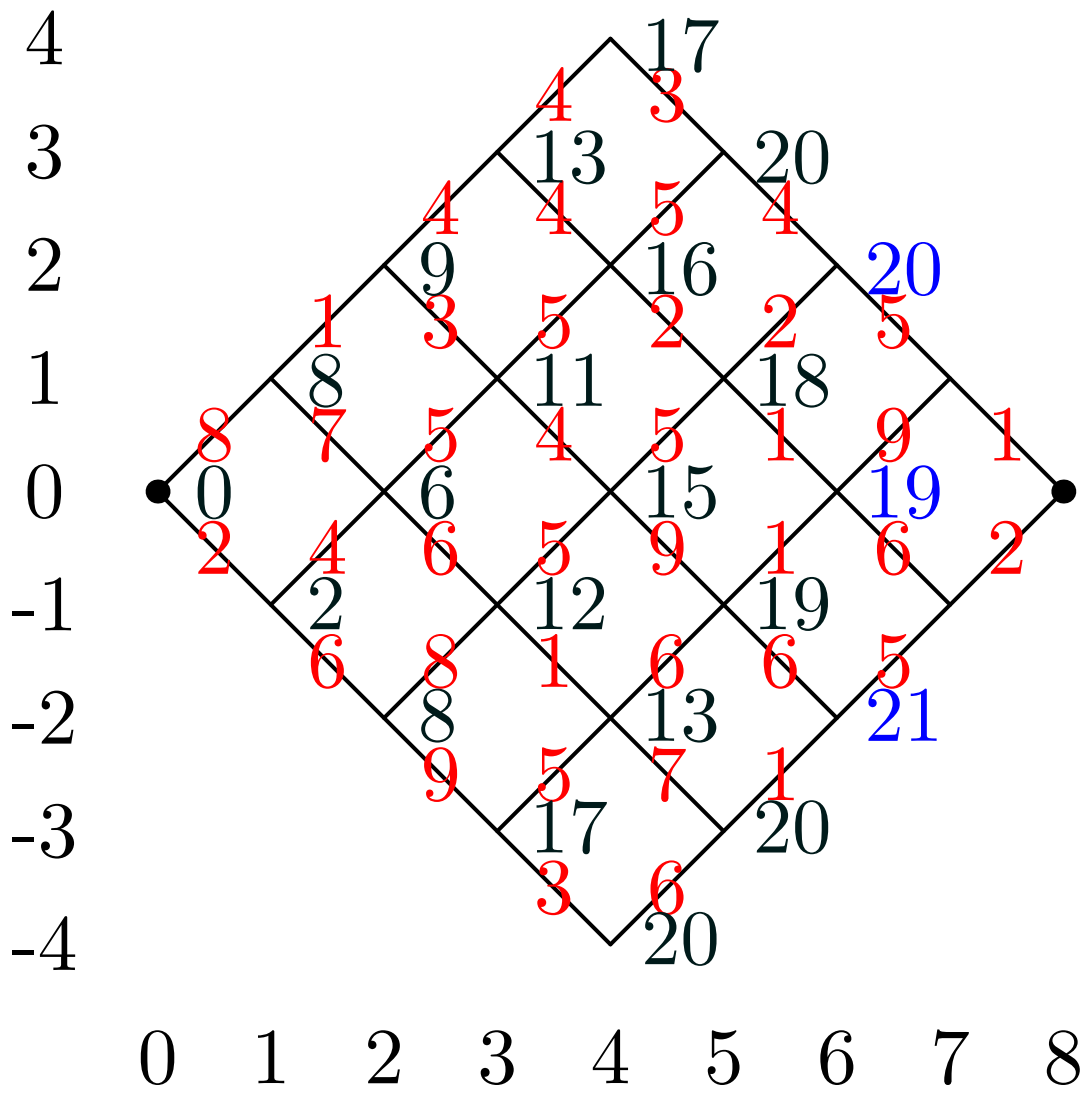
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13
4 4 5 4
9 16
1 3 5 2 2 5
8 11
8 7 5 4 5 1 9 1
0 6 15
2 4 6 5 9 1 6 2
2 12
6 8 1 6 6 5
8 13
9 5 7 1
17
3 6
20

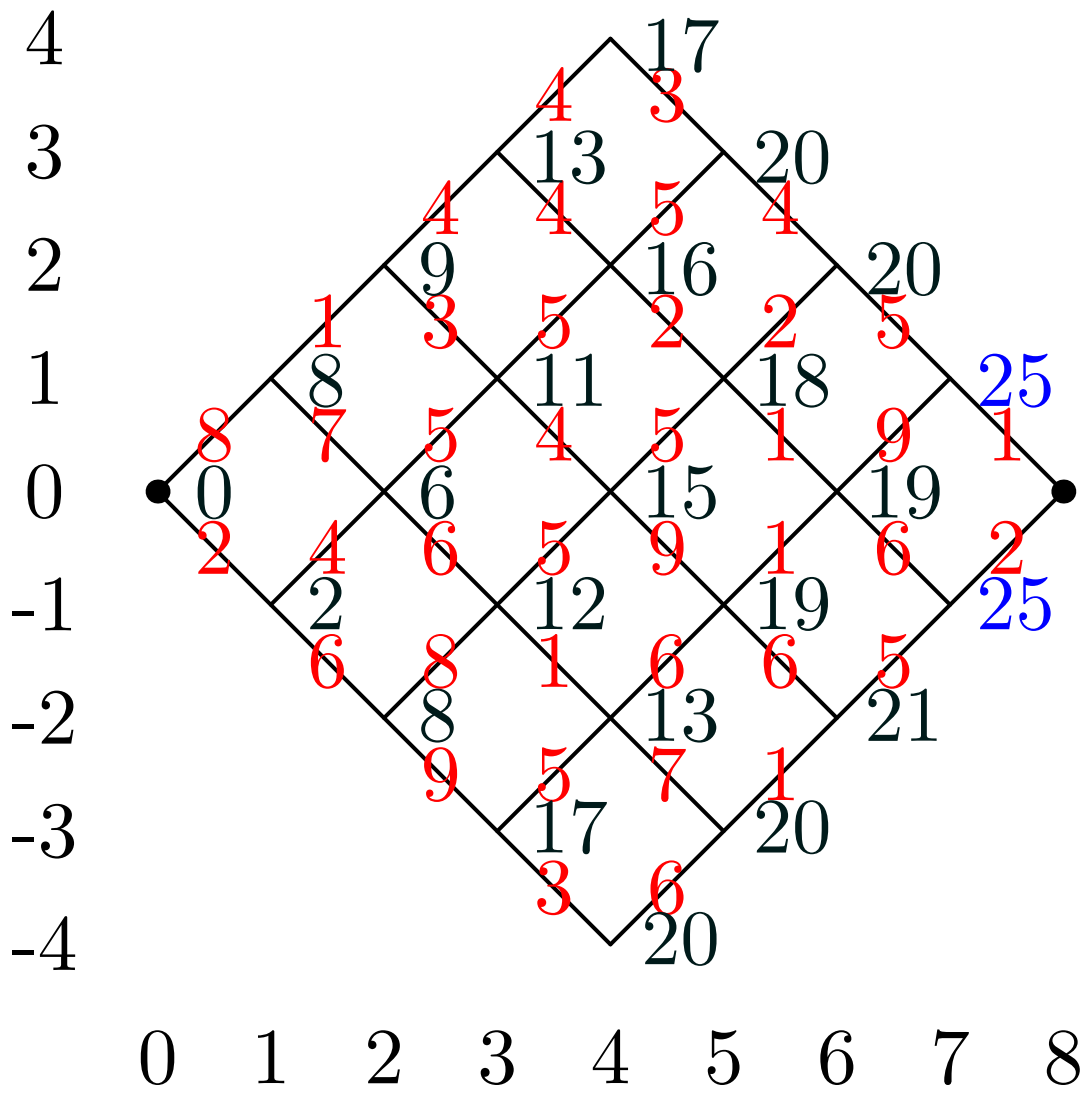
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13
20
4 4 5 4
9
16
1 3 5 2 2 5
8
11
18
8 7 5 4 5 1 9 1
0
6
15
2 4 6 5 9 1 6 2
2
12
19
6 8 1 6 6 5
8
13
9 5 7 1
17
20
3 6
20

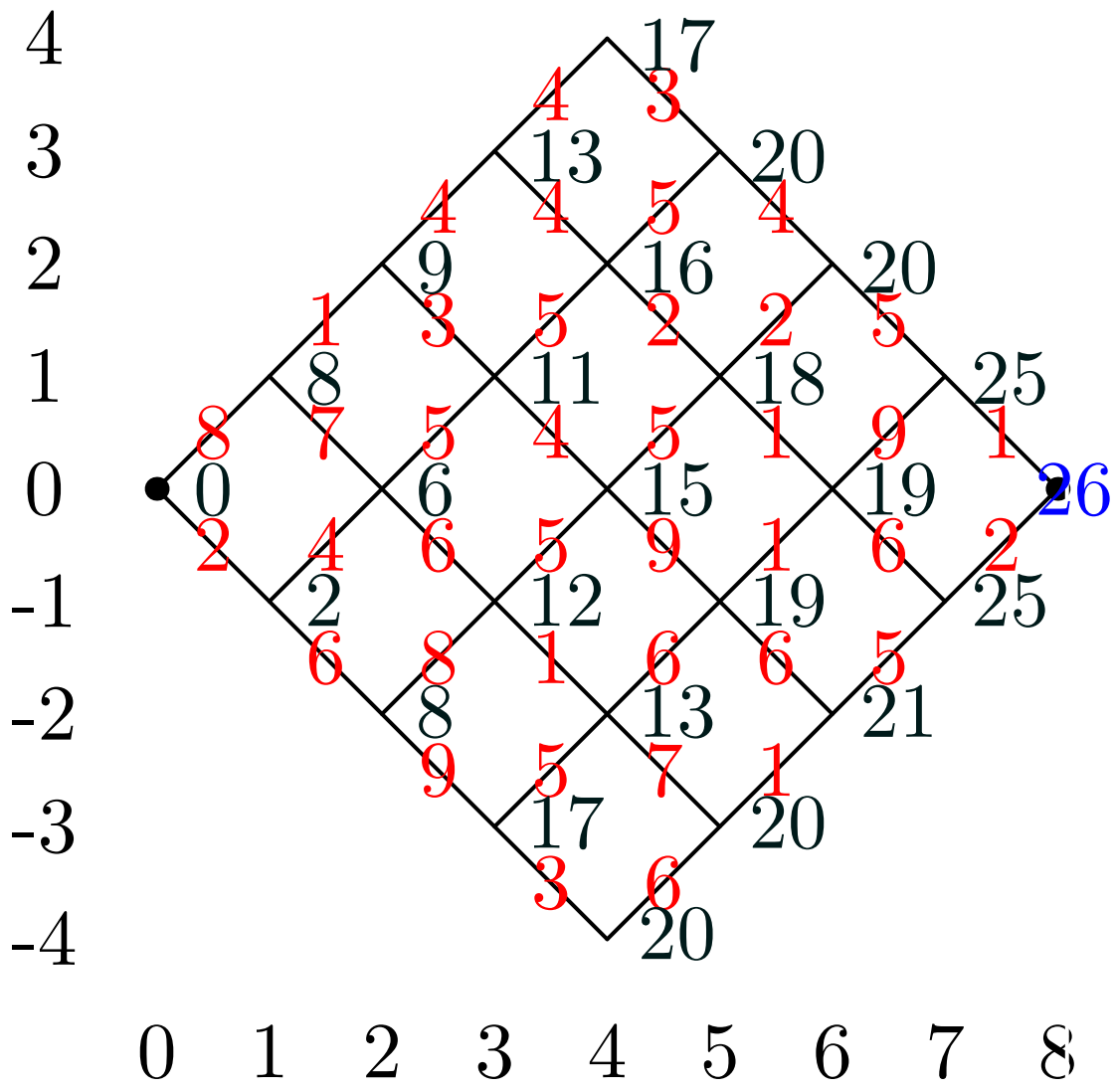
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18
8 7 5 4 5 1 9 1
0 6 15 19
2 4 6 5 9 1 6 2
2 12 19
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

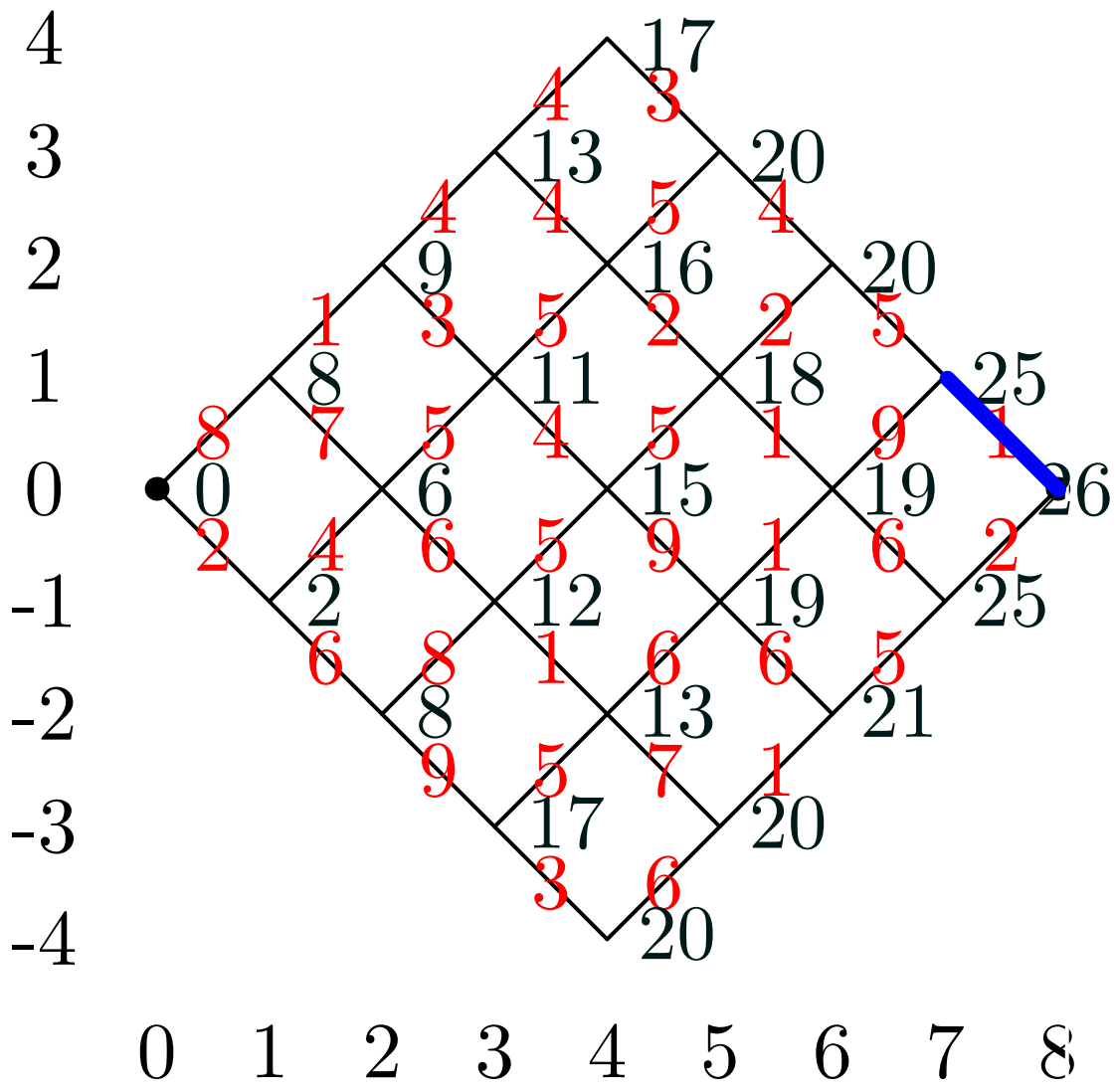
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

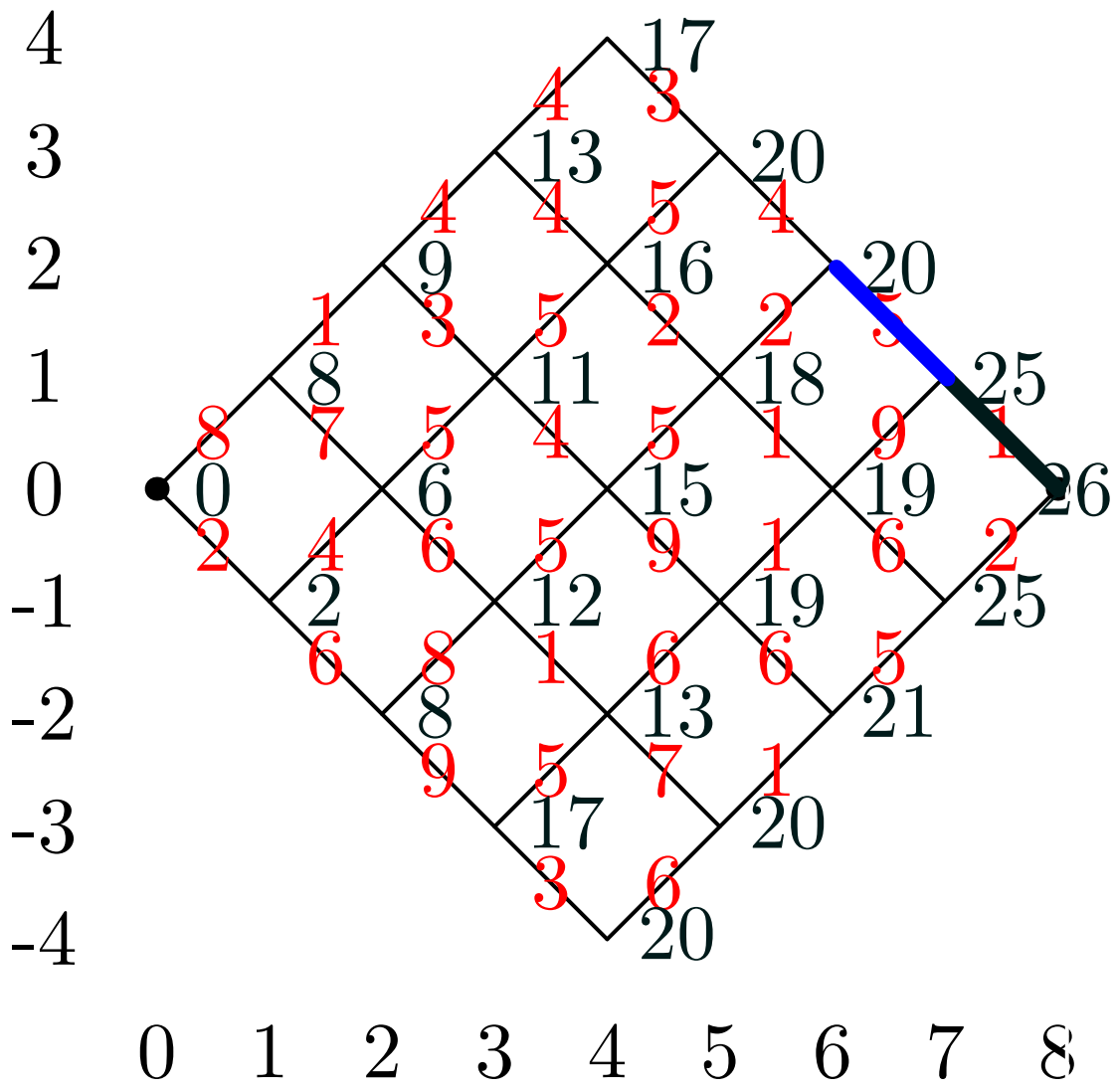
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

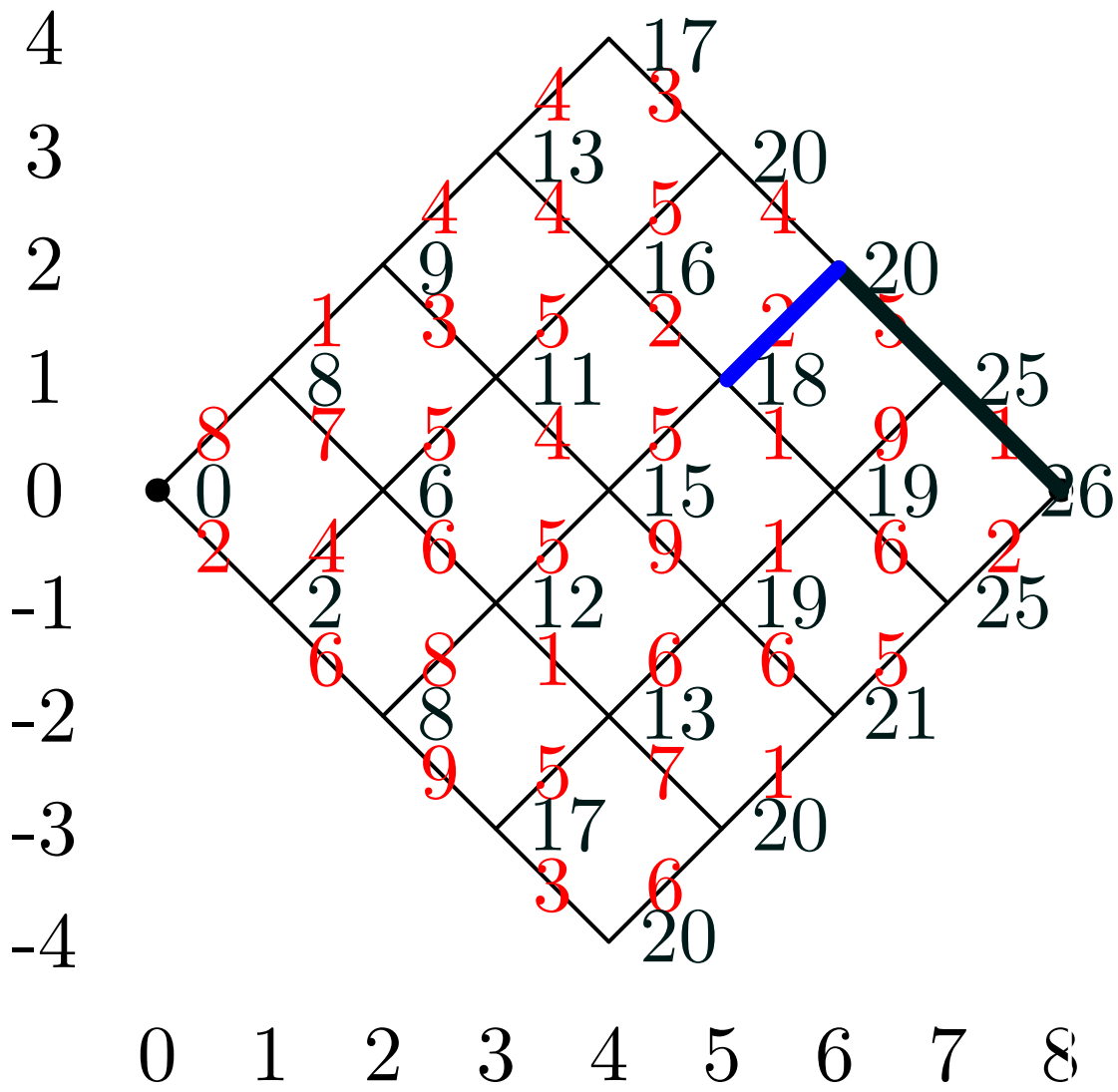
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

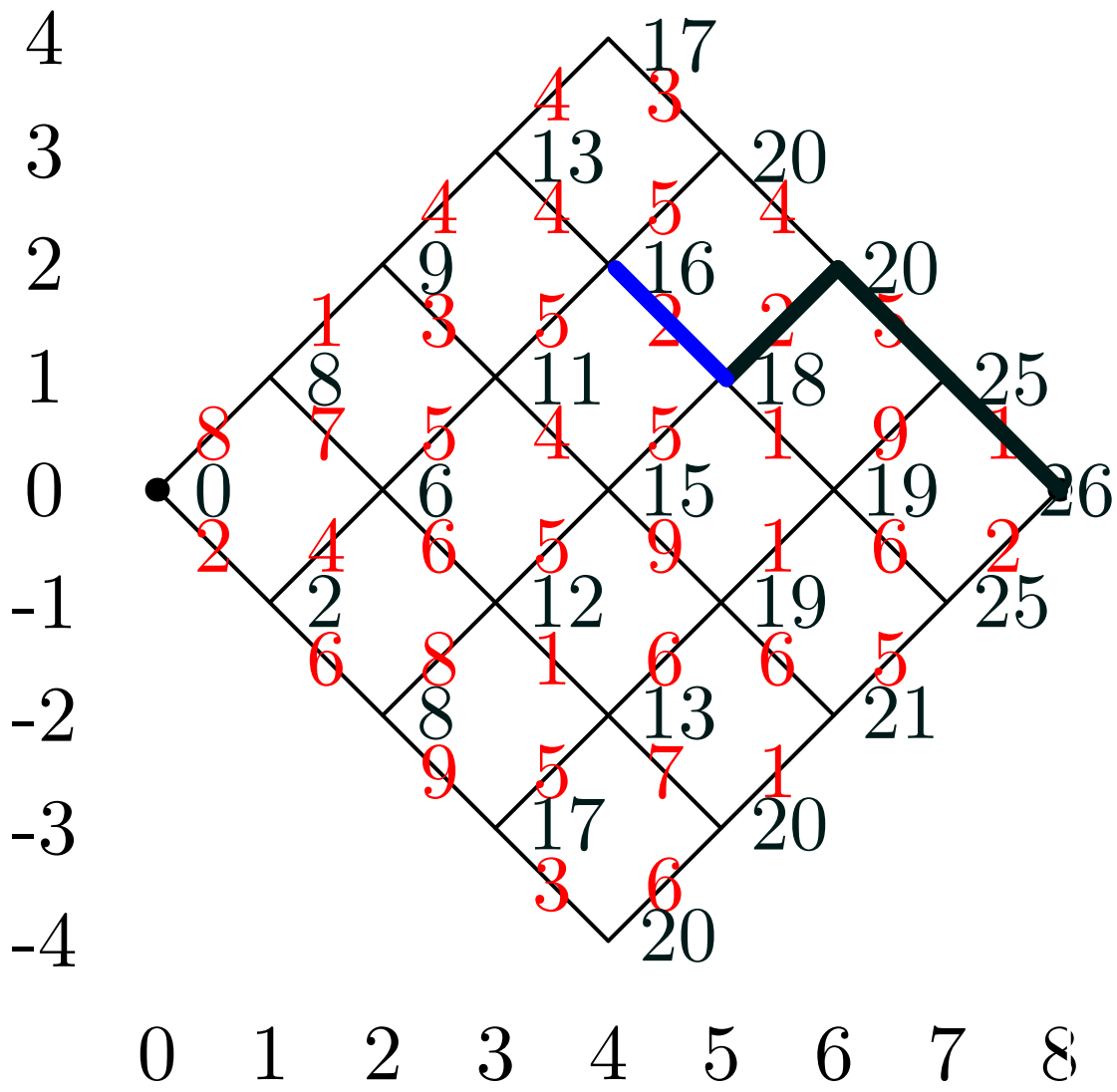
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 3
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

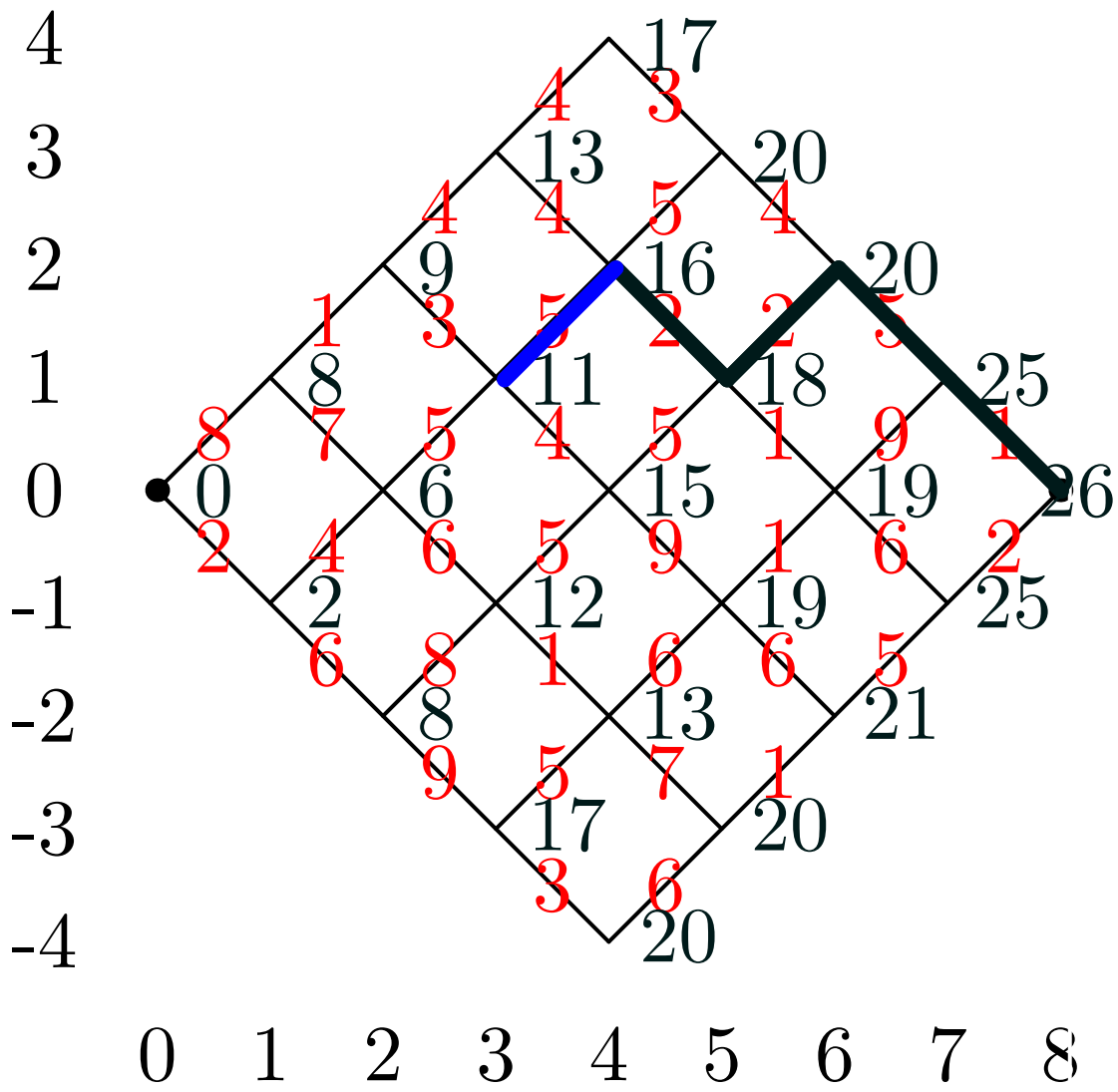
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 3
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

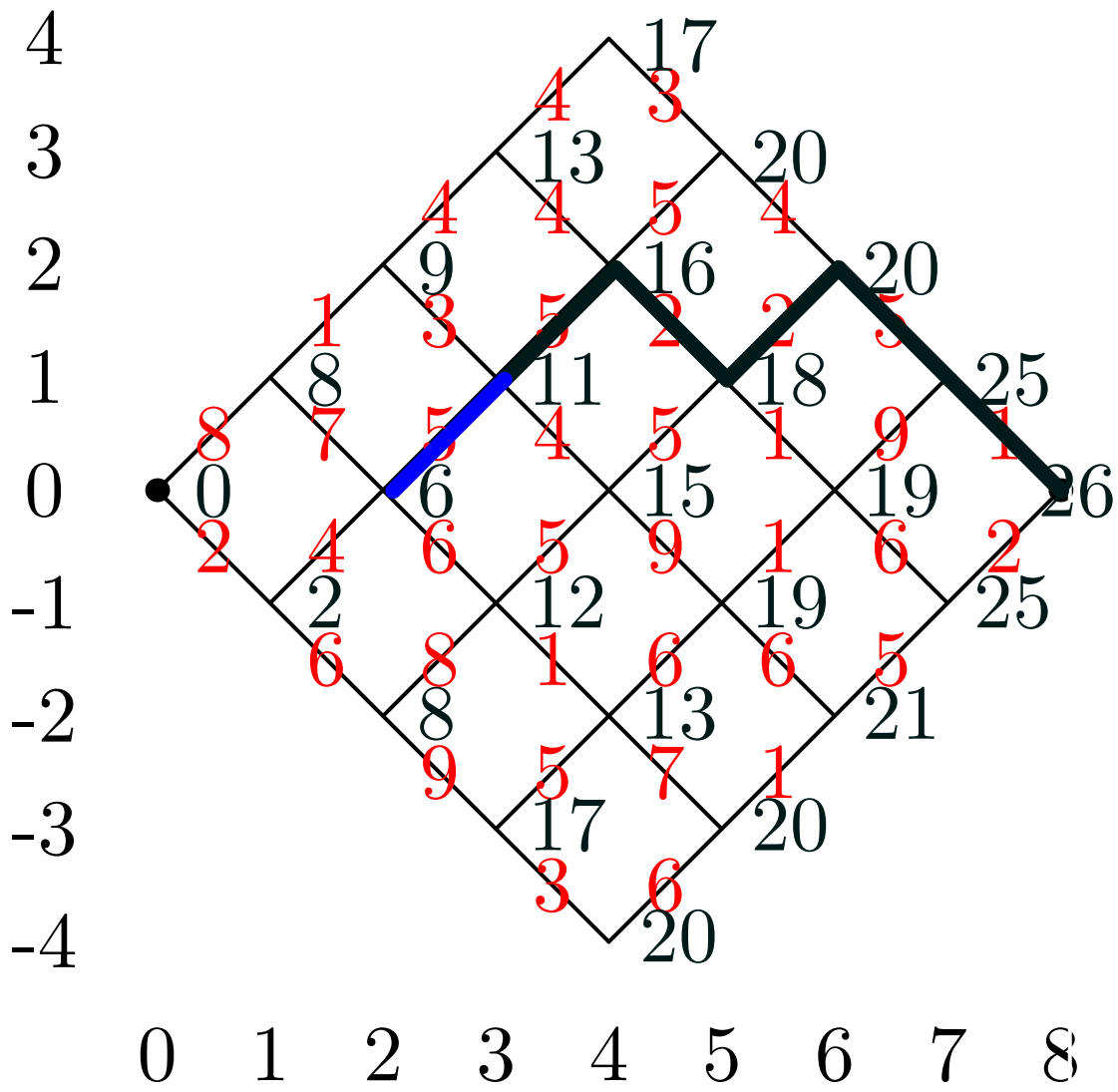
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

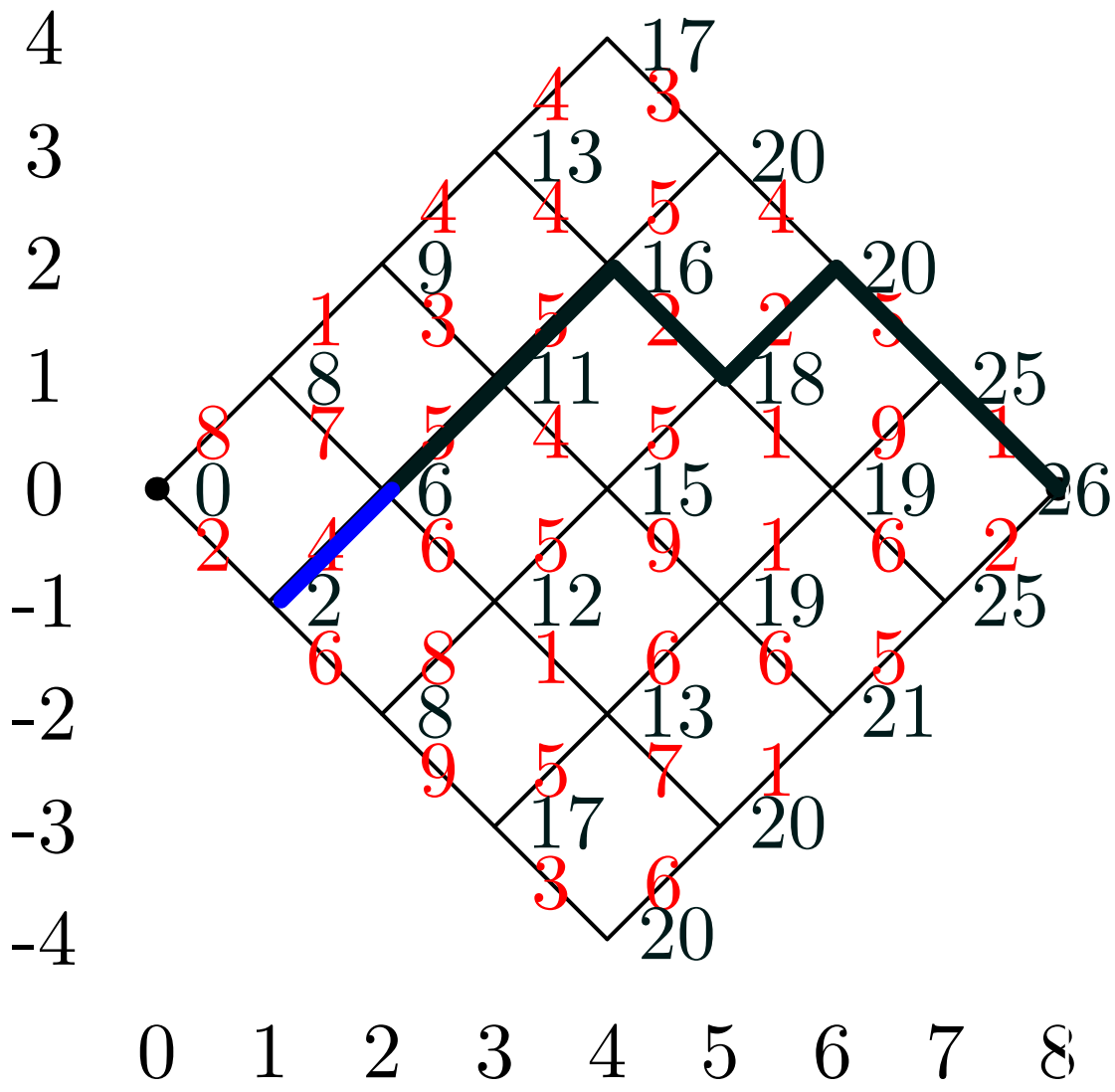
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

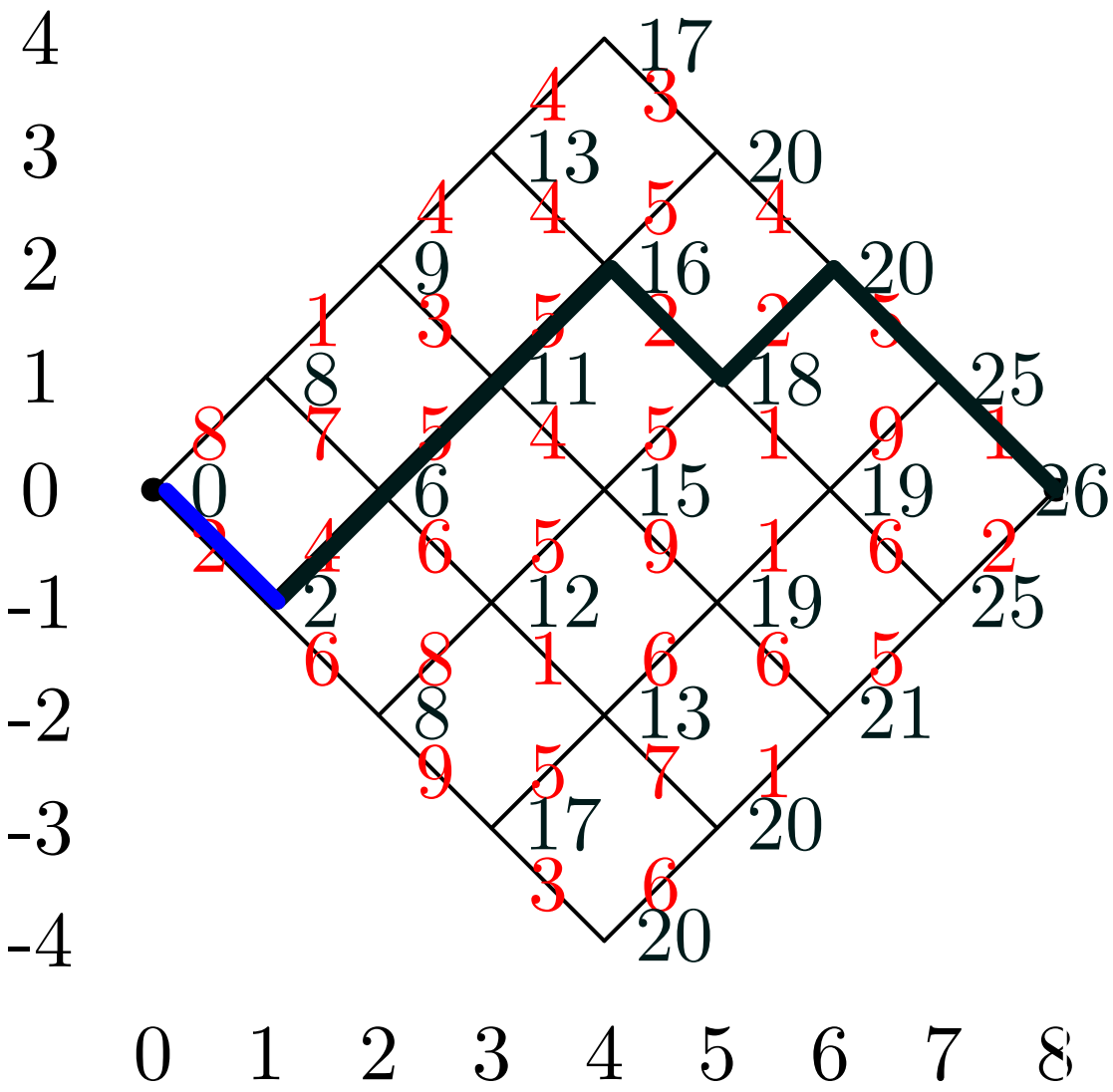
Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 3
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

Example


4
3
2
1
0
-1
-2
-3
-4
0 1 2 3 4 5 6 7 8
17
4 3
13 20
4 4 5 4
9 16 20
1 3 5 2 2 5
8 11 18 25
8 7 5 4 5 1 9 1
0 6 15 19 26
2 4 6 5 9 1 6 2
2 12 19 25
6 8 1 6 6 5
8 13 21
9 5 7 1
17 20
3 6
20

Backward Algorithm

- Having found the optimal costs $c_{(i,j)}$ we can find the optimal path starting from $(n, 0)$
- At each step we have a choice of going up or down
- We choose the direction which satisfies the constraint

$$c_{(i,j)} = c_{(i-1,j\pm1)} + w_{(i-1,j\pm1)(i,j)}$$

- If both directions satisfy the constraint we have more than one optimal path

Backward Algorithm

- Having found the optimal costs $c_{(i,j)}$ we can find the optimal path starting from $(n, 0)$
- At each step we have a choice of going up or down
- We choose the direction which satisfies the constraint

$$c_{(i,j)} = c_{(i-1,j\pm1)} + w_{(i-1,j\pm1)(i,j)}$$

- If both directions satisfy the constraint we have more than one optimal path

Backward Algorithm

- Having found the optimal costs $c_{(i,j)}$ we can find the optimal path starting from $(n, 0)$
- At each step we have a choice of going up or down
- We choose the direction which satisfies the constraint

$$c_{(i,j)} = c_{(i-1,j\pm1)} + w_{(i-1,j\pm1)(i,j)}$$

- If both directions satisfy the constraint we have more than one optimal path

Backward Algorithm

- Having found the optimal costs $c_{(i,j)}$ we can find the optimal path starting from $(n, 0)$
- At each step we have a choice of going up or down
- We choose the direction which satisfies the constraint

$$c_{(i,j)} = c_{(i-1,j\pm1)} + w_{(i-1,j\pm1)(i,j)}$$

- If both directions satisfy the constraint we have more than one optimal path

Time Complexity

- In our dynamic programming solution we had to compute the cost $c_{(i,j)}$ at each lattice point
- There were $(n/2+1)^2$ lattice point
- It took constant time to compute each cost so the total time to perform the forward algorithm was $\Theta(n^2)$
- The time complexity of the backward algorithm was $\Theta(n)$
- This compares with $\exp(\Theta(n))$ for the brute force algorithm

Time Complexity

- In our dynamic programming solution we had to compute the cost $c_{(i,j)}$ at each lattice point
- There were $(n/2+1)^2$ lattice point
- It took constant time to compute each cost so the total time to perform the forward algorithm was $\Theta(n^2)$
- The time complexity of the backward algorithm was $\Theta(n)$
- This compares with $\exp(\Theta(n))$ for the brute force algorithm

Time Complexity

- In our dynamic programming solution we had to compute the cost $c_{(i,j)}$ at each lattice point
- There were $(n/2+1)^2$ lattice point
- It took constant time to compute each cost so the total time to perform the forward algorithm was $\Theta(n^2)$
- The time complexity of the backward algorithm was $\Theta(n)$
- This compares with $\exp(\Theta(n))$ for the brute force algorithm

Time Complexity

- In our dynamic programming solution we had to compute the cost $c_{(i,j)}$ at each lattice point
- There were $(n/2+1)^2$ lattice point
- It took constant time to compute each cost so the total time to perform the forward algorithm was $\Theta(n^2)$
- The time complexity of the backward algorithm was $\Theta(n)$
- This compares with $\exp(\Theta(n))$ for the brute force algorithm

Time Complexity

- In our dynamic programming solution we had to compute the cost $c_{(i,j)}$ at each lattice point
- There were $(n/2+1)^2$ lattice point
- It took constant time to compute each cost so the total time to perform the forward algorithm was $\Theta(n^2)$
- The time complexity of the backward algorithm was $\Theta(n)$
- This compares with $\exp(\Theta(n))$ for the brute force algorithm

Outline

1. Why Align
2. Dynamic Programming
3. **Global Alignment**

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E A G A W G H E - E

- - P - A W - H E A E

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Global Matching

- We return to the problem of global matching
- In biology this is often known as the Needleman-Wunsch problem after the author that proposed the problem and a (not very efficient) solution
- We denote the cost of a substitution, $a \rightarrow b$, by $s(a, b)$
- For proteins we can take this to be our Blosum50 matrix
- There are different substitution matrices e.g. Blosum60 used for closely related species
- We will assume the cost of an insertion or deletion is -8

Problem Definition

- We are given two strings

$$\boldsymbol{a} = a_1\, a_2\, \ldots\, a_m \qquad\qquad \boldsymbol{b} = b_1\, b_2\, \ldots\, b_n$$

- We want to find the minimum edit distance measuring how to change $\boldsymbol{a}$ to $\boldsymbol{b}$ where the price of a substitution is $s(a, b)$ and of an insertion or deletion $d = -8$

- To compute this we compute a tableau showing the best possible moves using the dynamic programming formula

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

Problem Definition

- We are given two strings

$$\boldsymbol{a} = a_1\, a_2\, \ldots\, a_m \qquad\qquad \boldsymbol{b} = b_1\, b_2\, \ldots\, b_n$$

- We want to find the minimum edit distance measuring how to change $\boldsymbol{a}$ to $\boldsymbol{b}$ where the price of a substitution is $s(a, b)$ and of an insertion or deletion $d = -8$

- To compute this we compute a tableau showing the best possible moves using the dynamic programming formula

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

Problem Definition

- We are given two strings

$$\boldsymbol{a} = a_1\, a_2\, \ldots\, a_m \qquad\qquad \boldsymbol{b} = b_1\, b_2\, \ldots\, b_n$$

- We want to find the minimum edit distance measuring how to change $\boldsymbol{a}$ to $\boldsymbol{b}$ where the price of a substitution is $s(a, b)$ and of an insertion or deletion $d = -8$

- To compute this we compute a tableau showing the best possible moves using the dynamic programming formula

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

Problem Definition

- We are given two strings

$$\boldsymbol{a} = a_1\, a_2\, \ldots\, a_m \qquad\qquad \boldsymbol{b} = b_1\, b_2\, \ldots\, b_n$$

- We want to find the minimum edit distance measuring how to change $\boldsymbol{a}$ to $\boldsymbol{b}$ where the price of a substitution is $s(a, b)$ and of an insertion or deletion $d = -8$

- To compute this we compute a tableau showing the best possible moves using the dynamic programming formula

$$F(i,j) = \max \begin{cases} F(i-1, j-1) + s(a_i, b_j) \\ F(i-1, j) + d \\ F(i, j-1) + d \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0										
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0 ←	-8									
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0 ←	-8 ←	-16								
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0 ←	-8 ←	-16 ←	-24							
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0 ←	-8 ←	-16 ←	-24 ←	-32 ←	-40 ←	-48 ←	-56 ←	-64 ←	-72 ←	-80
P											
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8										
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2									
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9								
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17							
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A											
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16										
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10									
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3								
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4							
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W											
H											
E											
A											
E											

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	3	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	-5	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	1

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

E

E

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	-3	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	**3**	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	**-5**	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	**1**

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

- E

A E

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	-13	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	**-3**	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	**3**	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	**-5**	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	**1**

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

E - E

E A E

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	-5	**-13**	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	**-3**	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	**3**	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	**-5**	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	**1**

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E - E

H E A E

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	-8	-16	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	-17	-25	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	-20	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	**-5**	**-13**	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	**-3**	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	**3**	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	**-5**	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	**1**

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

G H E - E

- H E A E

Global Alignment

		H	E	A	G	A	W	G	H	E	E
	0	**-8**	**-16**	-24	-32	-40	-48	-56	-64	-72	-80
P	-8	-2	-9	**-17**	**-25**	-33	-41	-49	-57	-65	-73
A	-16	-10	-3	-4	-12	**-20**	-28	-36	-44	-52	-60
W	-24	-18	-11	-6	-7	-15	**-5**	**-13**	-21	-29	-37
H	-32	-14	-18	-13	-8	-9	-13	-7	**-3**	-11	-19
E	-40	-22	-8	-16	-16	-9	-12	-15	-7	**3**	-5
A	-48	-30	-16	-3	-11	-11	-12	-12	-15	**-5**	2
E	-56	-38	-24	-11	-6	-12	-14	-15	-12	-9	**1**

$$F_{i,j} = \max \begin{cases} F_{i-1,j-1} + s(a_i, b_j) \\ F_{i-1,j} - 8 \\ F_{i,j-1} - 8 \end{cases}$$

H E A G A W G H E - E

- - P - A W - H E A E

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Space Complexity

- For two strings of length m and n the time complexity of dynamic programming is the size of the tableau $O(m\,n)$
- This is massively faster than exhaustive search
- Unfortunately in biology we often want to match two very long strings (particularly when looking at DNA)
- The space complexity is then more of an issue than time complexity (we don't have enough memory to store the tableau)
- Algorithms exist that use linear space complexity
- Can still be too slow for DNA

Summary

- Comparing sequences is done routinely to understand function and evolution
- The dynamic programming strategy is widely used for inexact matching
- For global matching the dynamic programming solution is really quite easy
- We will also see that it is readily extendable

Summary

- Comparing sequences is done routinely to understand function and evolution
- The dynamic programming strategy is widely used for inexact matching
- For global matching the dynamic programming solution is really quite easy
- We will also see that it is readily extendable

Summary

- Comparing sequences is done routinely to understand function and evolution
- The dynamic programming strategy is widely used for inexact matching
- For global matching the dynamic programming solution is really quite easy
- We will also see that it is readily extendable

Summary

- Comparing sequences is done routinely to understand function and evolution
- The dynamic programming strategy is widely used for inexact matching
- For global matching the dynamic programming solution is really quite easy
- We will also see that it is readily extendable